

烟幕干扰弹的有效投放策略及参数计算

摘 要

随着精确制导武器防御需求的提升,无人机协同烟幕干扰弹投放策略成为高价值目标防护的关键技术。本文针对多场景烟幕干扰任务,建立了基于空间离散化采样和运动学分析的动态遮蔽模型,通过遗传算法优化投放参数,实现了对来袭导弹的有效干扰时长最大化。

对于问题一,本文建立了烟幕干扰弹和导弹 M1 的运动学模型和真目标圆柱体的空间离散化采样模型,利用几何视线分析算法进行有效遮蔽的判断,计算出有效遮蔽时长。首先,对真目标圆柱体的上下底面和侧面进行等间距的空间离散化采样,获取了 550 个空间离散化采样点。其次,运用运动学公式,在每一个时间离散化采样点实时更新导弹 M1 和烟幕干扰弹的位置,并判断真目标的空间离散化采样点与导弹 M1 的连线是否均穿过烟幕,进而判断是否构成有效遮蔽,最终求解出烟幕干扰弹对 M1 的有效遮蔽时长为 1.3916s。

对于问题二,本文在问题一的基础上,建立了决策变量为单无人机飞行方向、飞行速度、烟幕干扰弹投放时间、起爆延迟时间的 4 维待优化目标函数模型,并利用遗传算法求解出了最大有效遮蔽时长为 4.588s。此外,本文通过对上述四个决策变量的灵敏度分析发现了有效遮蔽时长对无人机飞行速度的敏感度显著低于其余三者。

对于问题三,本文在前两问的基础上引入了多弹互补遮蔽机制,建立了以飞行方向、飞行速度、3 个投放时刻和 3 个起爆延迟时间为决策变量的 8 维待优化目标函数模型,并利用遗传算法给出了烟幕干扰弹的最佳投放策略,求解出了最大有效遮蔽时长为 6.400s。

对于问题四,本文在问题一的基础上,建立了多个无人机和导弹 M1 的运动模型,并利用了问题三中的多弹互补遮蔽机制和遗传算法,解决了决策变量为 3 架无人机各自的飞行方向、飞行速度、烟幕干扰弹投放时间、起爆延迟时间的 12 维目标函数优化问题,求解出最大有效遮蔽时长为 12.45s,得到了烟幕干扰弹的最佳投放策略。

对于问题五,本文在前四问的基础上,完善了多导弹情况下有效干扰的判定模型。只要存在至少一枚导弹被完全遮蔽,该时刻即被计入有效干扰时间。同时,考虑到同一时刻下被有效遮蔽的导弹数量与整体干扰效能呈正相关,即遮蔽的导弹越多,对真目标的整体保护效果越强,因此,此 40 维待优化目标函数为三枚导弹各自被有效遮蔽的时长总和。利用遗传算法,求解出最大有效遮蔽时长为 20.24s,得到了烟幕干扰弹的最佳投放策略。

本文构建的模型兼具严谨性与实用性,空间离散化采样提升了遮蔽判定精度,遗传算法的改进策略有效缓解了高维变量下的局部最优问题。在文章的最后,也针对被忽略的一些因素进行了总结和思考,分析了模型的优缺点。

关键词: 多无人机协同 运动学建模 遗传算法 空间离散化采样 有效遮蔽

一、问题重述

1.1 问题背景

随着现代战争中对精确制导武器拦截需求的日益增长，如何通过非杀伤性干扰手段有效保护高价值目标成为亟待解决的课题。烟幕干扰技术凭借其快速响应、低成本及可重复部署的优势，已成为战场目标防护的重要技术路径。因此，针对无人机协同烟幕干扰弹的投放策略研究日益受到重视。此类研究聚焦于通过优化无人机飞行轨迹、投放时机及弹体起爆参数，构建动态复杂环境下多目标干扰的数学模型，为高价值目标防护的战术决策提供科学依据，从而提升战场生存能力[6]。

1.2 问题重述

针对无人机协同投放烟幕干扰弹保护真目标的任务需求，需构建多约束下的投放策略模型以最大化有效遮蔽时间。其中 5 架无人机初始位置已知，每架可挂载烟幕干扰弹并按需调整航向，确定后按匀速直线飞行，每架至少间隔 1 秒投放两枚干扰弹。干扰弹起爆后形成球状云团，以 $3m/s$ 匀速下沉，中心 10 米范围内 20 秒内可有效遮蔽真目标。来袭 3 枚导弹初始位置已知，以 $300m/s$ 速度直指假目标。任务要求通过设计无人机飞行方向、速度、干扰弹投放点及起爆点等策略，在导弹突防过程中形成动态烟幕遮蔽，最终实现多枚干扰弹对真目标的有效遮蔽时长最大化。

问题一：针对导弹 M1，使用无人机 FY1 投放 1 枚烟幕干扰弹实施干扰。假设 FY1 以 $120m/s$ 的速度朝假目标的方向匀速飞行，在受领任务后 1.5 秒时投放干扰弹，并在投放后 3.6 秒起爆。需计算该干扰弹对 M1 的有效遮蔽持续时间。

问题二：在问题一基础上增加策略优化维度，要求自主确定无人机 FY1 的最优飞行方向、速度及弹体投放、起爆点的参数组合，以最大化对导弹 M1 的遮蔽持续时间。

问题三：扩展至多弹协同投放场景，要求同一无人机 FY1 连续投放 3 枚烟幕干扰弹构建复合遮蔽体系。在此条件下确定投放策略，最大化对导弹 M1 的遮蔽持续时间。

问题四：引入多无人机协同机制，需要统筹 FY1、FY2、FY3 三机协同执行对 M1 的干扰任务，每个无人机各投放 1 枚烟幕干扰弹。在此条件下确定投放策略，最大化对导弹 M1 的遮蔽持续时间。

问题五：面对多目标防御需求，需制定全域协同投放策略应对 M1、M2、M3 三枚导弹的协同突防。在拥有 5 架无人机，且单机架次最多 3 弹的资源约束下，确定投放策略，最大化对 3 枚导弹的遮蔽持续时间。

二、问题分析

2.1 问题一的分析

为了求解烟幕干扰弹对导弹 M1 的有效遮蔽时长，需要建立包含空间遮蔽条件和时间演化的数学模型。首先将圆柱形真目标表面离散化为若干采样点，有效遮蔽的条

件定义为：导弹 M1 与所有采样点之间的视线均被半径为 10m 的烟幕球阻断。

在运动学建模方面，无人机 FY1 以 $120m/s$ 匀速飞向假目标，1.5 秒后在投放点释放干扰弹，干扰弹随后经历 3.6 秒的平抛运动（考虑重力加速度为 $9.8m/s^2$ ）到达起爆点。与此同时，导弹 M1 以 $300m/s$ 沿直线飞向假目标。起爆后烟幕云团以 $3m/s$ 匀速下沉，持续 20 秒有效时间。通过离散化时间步长，在每个时间节点计算更新后的导弹位置和烟幕中心坐标，并验证所有采样点视线是否被烟幕球阻断，最终统计满足条件的时间段总和即为有效遮蔽时长。

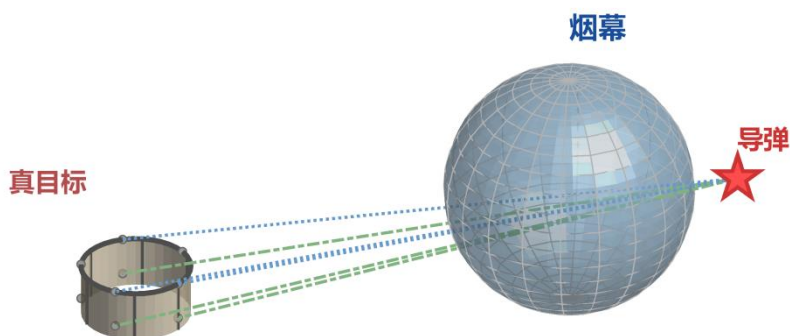


图 1 球状烟幕对于导弹发现真目标的有效遮挡示意图

2.2 问题二的分析

问题二需要确定无人机 FY1 的最优飞行方向、飞行速度、烟幕干扰弹投放点和起爆点，以最大化遮蔽时间。这本质上是一个四变量的联合优化问题。在实际分析中，烟幕干扰弹投放点和起爆点可以转化为两个时间参数：投放前的匀速运动时间和投放后至起爆前的运动时间。因此，最终的决策变量包括：飞行方向、飞行速度、投放前的匀速运动时间以及投放后至起爆前的运动时间。

基于问题一建立的有效遮蔽时长计算模型，我们可以计算任意给定决策变量组合对应的有效遮蔽时长。由于该优化问题具有以下特点：目标函数没有解析表达式，决策变量较多，且可能存在多个局部最优解，因此我们选择遗传算法作为求解方法。遗传算法作为一种智能优化算法，能够有效处理这类复杂的多变量单目标优化问题。

2.3 问题三的分析

问题三在问题一、二的基础上，进一步扩展为单架无人机 FY1 为干扰单枚来袭导弹 M_1 投放的多枚烟幕干扰弹（3 枚）的投放策略优化问题。与问题二中仅需优化单枚干扰弹的投放参数不同，本问题需联合优化 3 枚干扰弹的投放顺序、时间间隔与空间分布，以实现遮蔽时间最大化，且允许多段遮蔽效果之间存在时间间隙。

由于每架无人机最多可挂载并投放 3 枚烟幕干扰弹，且根据题设要求，相邻两枚干扰弹的投放时间间隔不得小于 1 秒。因此，本问题的决策变量不仅包括每枚干扰弹的投放参数（即飞行方向、飞行速度、投放前匀速运动时间、投放后起爆延迟时间），

还需考虑各枚干扰弹之间的投放时序关系,故可将问题转化为一个**多阶段、带时间耦合约束的序列优化问题**。

在实际建模中,我们固定了无人机的飞行方向与速度在整个任务过程中保持不变,而仅对 3 枚干扰弹各自的**投放时刻与起爆延迟时间**进行优化。这样,每枚干扰弹的投放点与起爆点将由其对应的投放时刻与延迟时间唯一确定,从而将原本高维的混合变量空间降维为以**飞行方向、飞行速度、3 个投放时刻和 3 个起爆延迟时间**为决策变量的**8 维优化问题**。

由于目标函数——**3 枚干扰弹共同作用下的总有效遮蔽时长**,没有显式解析表达式,且关于决策变量呈现强非线性、非凸性,并可能存在多个局部最优解;同时,目标函数的解空间维度较高,传统梯度类或枚举类方法难以高效求解。因此,仍采用**遗传算法**作为求解工具。

2.4 问题四的分析

问题四在问题三基础上扩展为三架无人机协同干扰任务。与单机优化不同,需同时协调多机的飞行参数(方向角、速度)和干扰弹投放参数(投放时间、起爆延迟),决策变量增至 **12 维**。各无人机初始位置不同,其飞行路径与投放时机存在耦合关系,需从系统层面进行联合优化。多机协同可利用空间异构性形成分布式遮蔽网络,扩大覆盖范围。目标函数仍是最大化总有效遮蔽时长,但**遮蔽判定标准改为三团烟幕云的联合阻断效果,需同时遮挡所有采样视线**。鉴于变量维度高、耦合性强,采用遗传算法进行求解。算法将 12 维变量编码为染色体,在满足约束条件下演化出最优协同策略,实现干扰效能最大化。

2.5 问题五的分析

问题五需要求解多无人机协同投放多枚烟幕弹以干扰多枚导弹的最优策略,该问题可视为问题三与问题四在多变量优化框架下的综合推广。其核心目标函数为烟幕弹系统对多枚导弹的总有效干扰时长,我们将其**定义为每枚导弹有效遮蔽时长的总和**。基于问题三和问题四已建立的有效遮挡判定模型,可计算在给定烟幕弹投放策略下各导弹的有效遮蔽时长,求和即得待优化的目标函数。在此基础上,可采用遗传算法进行优化求解。需要指出的是,本问题中待优化变量多达 40 个,显著多于前述问题,因此算法容易陷入局部最优。为提升搜索效率与解的质量,需对遗传算法实施多维度改进,以增强其全局探索能力与优化精度。

三、模型假设

- 1、导弹出现后，警戒雷达即刻发现，并将任务派给无人机。
- 2、无人机飞行方向的调整是瞬时完成的。
- 3、只要导弹和真目标表面上的某个点所构成连线不经过烟幕，就不构成有效遮挡。
- 4、烟雾干扰弹起爆后 20 秒内，烟幕浓度保持不变，20 秒后烟幕浓度即刻降为 0。
- 5、起爆瞬间，烟幕的球心坐标等于起爆前干扰弹的坐标。
- 6、运动中的导弹可以建模为一个点。
- 7、当多枚干扰弹共同遮挡导弹时,认为此时多枚干扰弹都记录有效时间。

四、符号说明

符号	说明	单位
t_1	无人机从接收任务至投放烟幕干扰弹所用时间	s
t_2	烟幕干扰弹从投放到起爆所用时间	s
$\vec{v}_1$	导弹的速度向量	m/s
$\vec{v}_2$	无人机的速度向量	m/s
$\vec{v}_3$	烟幕云团形成后的下沉速度向量	m/s
$A_{m,n}$	真目标圆柱上的第 m 条母线的第 n 个采样点	无
t_{mn}	$A_{m,n}$ 点被遮挡的时间区间	s
θ	无人机飞行速度与 x 轴正方向的夹角	度 ($^{\circ}$)
T	真目标被有效遮挡的总时长	s

五、模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 模型的建立

圆柱形目标的多点离散采样模型

已知目标圆柱半径 7 m、高 10 m，其几何尺度与烟幕干扰弹爆炸形成的球形云团相近。为了突破“中心点唯一判别”所带来的精度瓶颈，本文提出基于均匀离散采样的精细建模方法：

- 1、在上底面圆周上以弧长等距方式均匀选取 50 个采样点；
- 2、过每个采样点作圆柱母线（竖直方向直线），该母线与下底面圆周相交，形成 50 条等距母线；
- 3、在每条母线上再按高度等距采样 11 个点，即每隔 1 米采样 1 个点；
- 4、第 m 条母线（ $m = 1, 2, \dots, 50$ ）第 n 个采样点（ $n = 1, 2, \dots, 11$ ）记为 $A_{m,n}$ 。

通过上述步骤，可以在圆柱表面得到 550 个均匀的空间采样点。对每个 $A_{m,n}$ 独立进行遮挡判断，显著提升了模型对复杂烟幕干扰场景的刻画精度与计算可靠性。

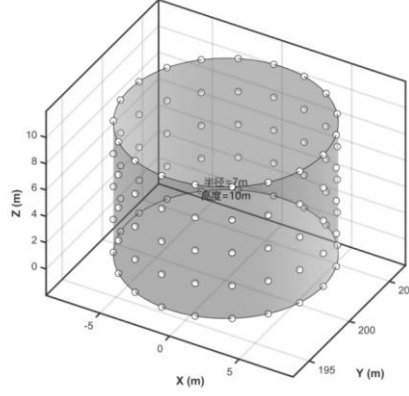


图 2 圆柱形目标的多点离散采样示意图

导弹以及烟幕干扰弹运动模型

为便于建模与后续分析，定义无人机接收任务指令的时刻为时间零点 $t = 0$ 。根据任务执行过程的自然划分，可将整个时间轴划分为三个连续阶段：

阶段一：从无人机接收任务至投放烟幕干扰弹之前；

阶段二：从烟幕干扰弹投放至其起爆之间；

阶段三：烟幕干扰弹起爆之后。

设导弹在任意时刻的位置为 $M_1(x_1, y_1, z_1)$ ，无人机 FY1 上的烟幕干扰弹中心点为 $F(x_f, y_f, z_f)$ 。定义如下时间参数和速度向量：

t_1 ：无人机从接收任务至投放烟幕干扰弹所用时间；

t_2 ：烟幕干扰弹从投放到起爆所用时间；

$\vec{v}_1$ ：导弹的速度向量；

$\vec{v}_2$ ：无人机的速度向量；

$\vec{v}_3$ ：烟幕云团形成后的下沉速度向量；

$\vec{e}_z$ ：竖直向上的单位方向向量。

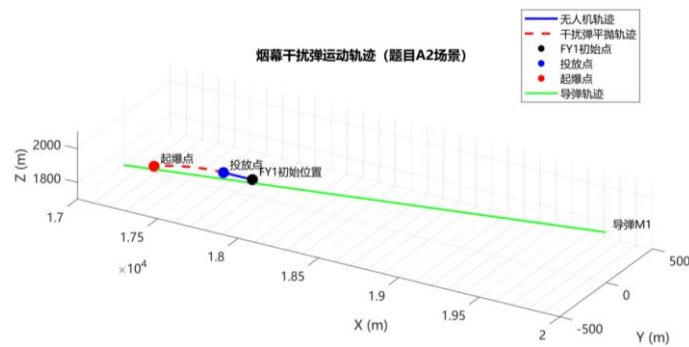


图 3 烟幕干扰弹及导弹运动轨迹

则各阶段运动模型可分别描述如下：

1、导弹运动模型

在整个过程中，导弹保持匀速直线飞行，其位置随时间变化满足：

$$M_1(x_1, y_1, z_1) = M_{10}(x_{10}, y_{10}, z_{10}) + t \times \vec{v}_1$$

其中， M_{10} 为导弹在初始时刻的位置。

2、烟幕干扰弹运动模型

烟幕干扰弹的运动过程需考虑无人机运动、重力作用及起爆后的烟幕云团沉降行为，其位置随时间变化表达式为：

$$F(x_f, y_f, z_f) = F_0(x_{f0}, y_{f0}, z_{f0}) + t_1 \times \vec{v}_2 + t_2 \times \vec{v}_2 - \vec{e}_z \times \frac{1}{2}gt_2^2 + \vec{e}_z \times (t - t_1 - t_2) \times \vec{v}_3$$

其中， F_0 为干扰弹初始位置， g 为重力加速度。

该模型综合考虑了无人机平台运动、干扰弹自由落体运动及烟幕云团形成后的沉降特性，能够较为准确地反映干扰弹在三维空间中的轨迹变化，为后续干扰效果评估提供良好基础。

有效遮蔽判断模型

为评估烟幕云团对导弹导引视线的遮蔽效果，本文提出一种基于空间几何关系的有效遮蔽判断模型。计算烟幕云团中心点 F 与目标圆柱表面每个采样点 $A_{m,n}$ 及导弹位置 M_1 构成的视线段之间的空间距离，判断烟幕云团是否对该视线构成有效遮挡。

设单位方向向量 $\vec{n}_{m,n}$ 为从导弹 M_1 指向采样点 $A_{m,n}$ 的方向向量，定义烟幕中心点 F 到视线段 M_1A_{mn} 的空间距离为 $d_{m,n}$ 。通过向量投影法，可得：

$$d_{m,n} = \sqrt{||\vec{M_1F}||^2 - (\vec{M_1F} \cdot \vec{n}_{mn})^2}$$

若 $d_{m,n} \leq r$ （其中 r 为烟幕云团的半径），则认为烟幕对该视线段构成潜在遮挡。进一步，为准确判断遮挡是否有效，引入几何约束条件：垂足点 P 必须位于线段 $M_1A_{m,n}$ 上，或当 P 不在线段上时，需确保 $||\vec{M_1F}|| \leq r$ ，以避免误判。

对所有采样点 $A_{m,n}$ 进行上述判断，记录满足条件的时间段 $t_{m,n}$ ，最终取所有时间段之交集，得到有效遮蔽时间：

$$T = \bigcap_{m=1}^{50} \bigcap_{n=1}^{11} t_{mn}$$

该模型通过离散采样与几何判断相结合，能够较为准确地评估烟幕干扰在三维空间中的实际遮蔽效果，为后续干扰策略优化提供理论依据。

5.1.2 模型的求解

1、模拟运动过程

本问中，无人机 FY1、导弹 M_1 和烟幕弹的运动路径均已确定，需要模拟它们的运动过程，确定它们在每个时刻的位置并判断是否形成有效遮挡，从而得到烟幕对导弹产生有效干扰的持续时间。具体算法流程如下：

算法步骤

Step 1 初始化坐标值

- (1) 首先设定无人机 FY1 和导弹 M_1 初始坐标，然后计算它们在阶段一（无人机发现导弹到投出烟幕弹前）的坐标变化，本阶段无人机做水平直线匀速运动。
- (2) 随后令烟幕弹中心坐标继承此刻无人机坐标，再计算烟幕弹中心和导弹在阶段二（烟幕弹投出到爆炸前）的坐标变化，本阶段烟幕弹做平抛运动。

由此我们得到了在开始小步长遍历判断每个时刻是否形成有效遮挡前，即第 5.1 秒时，烟幕弹和导弹的位置。

Step 2 对圆柱形真目标进行离散采样

- (1) 在上底面圆周上以弧长等距方式均匀选取 50 个采样点；
- (2) 过每个采样点作圆柱母线（竖直方向直线），该母线与下底面圆周相交，形成 50 条等距母线，在每条母线上再按高度等距采样 11 个点，即每隔 1 米采样 1 个点，得到 550 个均匀的离散点。

Step 3 判断每个时刻是否形成有效遮挡

对于每个时刻：

- (1) 更新这个时刻烟幕弹中心和导弹的位置坐标；
 - (2) 依次将每个真目标表面上的离散点与导弹进行连线，判断连线的线段是否经过烟幕弹球体，只要有任意一条线段不经过，即认为此刻未形成有效遮挡。
-

通过预先输入的起始时刻、终止时刻和遍历步长，我们可以通过小步长遍历的方式确定形成有效遮挡的起始时间、终止时间和持续时长。

特别需要注意的一点是，判断真目标被烟幕所遮挡的依据是球心到导弹与离散点连线的距离不超过球体半径（10m），而不是球心到导弹与离散点所在直线的距离。如果计算的是点到直线的距离，会导致答案异常偏大。

2、求解结果及分析

通过上述求解步骤，可以计算出烟幕干扰弹对 M_1 的有效遮蔽时长，以及有效遮蔽时段的开始和结束时刻，结果如下表所示：

有效遮蔽时长	有效遮蔽开始时刻 t_{start}	有效遮蔽结束时刻 t_{end}
--------	----------------------	--------------------

1.3916 秒	8.0566 秒	9.4482 秒
----------	----------	----------

注：我们认为雷达发现导弹的时刻为 $t = 0$ 。

5.2 问题二模型的建立与求解

5.2.1 模型的建立

目标函数的构建

在第一问建立的离散采样模型基础上，本问将无人机运动参数从固定值扩展为优化变量。考虑到无人机飞行方向对遮蔽效果的显著影响，我们引入极坐标参数化方法：

$$\vec{v}_2 = v_2 (\cos \theta \vec{e}_x + \sin \theta \vec{e}_y)$$

其中 $\theta \in [0, 2\pi]$ 为方位角， $v_2 \in [70, 140] \text{ m/s}$ 为速度模长。由此构建的遮蔽时长目标函数为：

$$T_{\max} = \max_{\theta, v_2, t_1, t_2} \left(\prod_{m=1}^{50} \prod_{n=1}^{11} t_{mn}(\theta, v_2, t_1, t_2) \right)$$

参数可行域的推导

在优化问题中，确定参数的可行域是确保求解过程有效且高效的关键步骤。针对无人机飞行方向、速度、投放时间和起爆延迟等参数，本文通过理论分析和物理约束条件，精确推导出各参数的可行范围，为后续的优化算法提供坚实的基础。

1. 最优飞行方向的参数优化范围

无人机 FY1 在接到指令后瞬时调整飞行方向，随后沿水平方向匀速飞行。因此，飞行方向的参数为飞行速度的方位角 θ ，即无人机 FY1 的飞行速度 $\vec{v}_2$ 与 x 轴正方向单位向量 $\vec{e}_x$ 的夹角。根据题意，方位角 θ 的取值范围为 $[0, 2\pi]$ ，覆盖了所有可能的水平飞行方向。

2. 无人机飞行速度的参数优化范围

无人机的飞行速度大小 v_2 是其性能的重要指标之一。根据任务需求和无人机的物理限制，飞行速度的优化范围设定为 $[70, 140] \text{ m/s}$ 。这一范围既满足了任务对速度的基本要求，又考虑了无人机的实际飞行能力。

3. 投放时间和起爆延迟

不难看出， t_1 的下界为 0s。考虑 t_1 的上限时，我们并不需要得到上确界，只要得到上界，即可利用遗传算法进行计算。为此，希望计算出 t_1 在什么取值范围下，烟幕弹永远不可能遮蔽导弹 M_1 的视野。不妨考虑最坏的一种情况，即在该 t_1 取值下，不论无

人机的速度方向和大小如何选取，烟幕弹引爆前，导弹已经完全越过烟幕弹的烟幕范围，这可由如下公式给出：

$$t_1 \leq \frac{x_{10} - x_f}{v_{1x} - v_2}$$

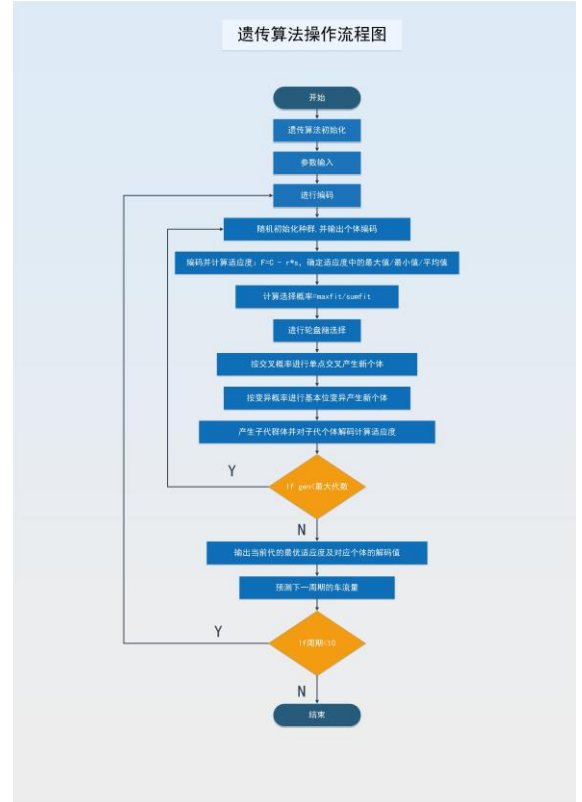
其中， x_{10} 是导弹 M_1 在 x 轴方向的初始坐标， x_f 是无人机 FY1 在 x 轴方向的初始坐标， v_{1x} 是导弹 M_1 在 x 轴方向的速度分量， v_2 是无人机 FY1 的速度大小。代入具体数据可得， $t_1 < 10s$ ，故将 t_1 的参数优化范围设置为 $[0,15]$ ，单位为秒(s)。

同理可得 t_2 的参数优化范围设置为 $[0,10]$ ，单位为秒 s。

4. 遗传算法参数的确定

遗传算法参数的确定基于问题特性和计算效率的综合考量。种群规模设为 300，既满足四维参数空间的搜索需求，又能控制计算成本。最大进化代数 50 代通过收敛曲线确定，超过 40 代后改进幅度已小于 2%。交叉率设置为 0.8。另外，我们采用自适应可行变异函数确保所有解满足时间参数的物理约束。

图 4 遗传算法基本操作流程



5. 2. 2 模型的求解

1. 遗传算法求解

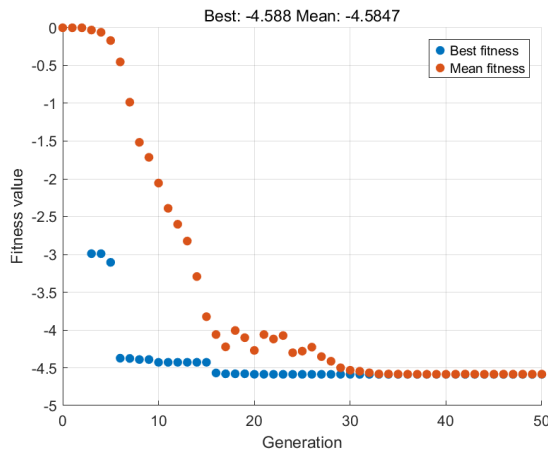


图 5 使用遗传算法迭代 50 次求解有效遮蔽时间

算法步骤

Step1 遗传算法搜索空间定义

(1) 适应度函数

$$\text{fitness}(\mathbf{x}) = -t_{\text{eff}}(\mathbf{x})$$

$t_{\text{eff}}(\mathbf{x})$ 为有效遮蔽总时长，GA 按最小化适应度进行演化。

(2) 设置种群规模 $N_{\text{pop}} = 100$ 、最大进化代数 $G_{\text{max}} = 30$ 、交叉概率 $p_c = 0.8$ 等参数，并指定约束边界。

Step2 遗传算法演化

调用 MATLAB 内置 GA 求解器，在满足边界约束条件下演化种群，输出最优个体 $\mathbf{x}^* = [\theta^*, v_{\text{FY}}^*, t_{\text{rel}}^*, t_{\text{lag}}^*]^T$ 及对应最大有效遮蔽时长 $t_{\text{eff}}^* = -\text{fitness}(\mathbf{x}^*)$

Step3 遮蔽过程精细化仿真

- (1) 以 1 ms 为步长遍历整个时间网格，首先更新导弹当前位置；若该时刻位于烟幕云生存周期内，则按下沉速度刷新云团中心坐标；
 - (2) 对假目标上 550 个关键点逐一进行视线相交判断，若所有点均被云团球体遮蔽，则将该时刻标记为有效；
 - (3) 合并连续有效区间并累加其长度，得到总有效遮蔽时长。
-

2. 求解结果及分析

通过上述求解步骤，可以计算出飞行方向、速度、投放时间、延迟时间、烟幕干扰弹的有效遮蔽时长，以及有投放点和起爆点坐标，结果如下表所示：

飞行方向角(角度)	飞行速度(m/s)	投放时间(s)	延迟时间(s)	有效遮蔽时间(s)	
5.02840813	132.457978	0.94834062	0.03742112	4.588	

投放点 X	投放点 Y	投放点 Z	起爆点 X	起爆点 Y	起爆点 Z
17925.13183	11.01013677	1800	17930.06948	11.44459216	1799.993138

5.2.3 模型参数敏感度分析

在模型求解过程中，我们发现使用遗传算法求解出的有效遮挡时间大致稳定在 4.58 秒到 4.59 秒之间，但每次计算得到的其他参数(如无人机飞行速度、飞行方向角、投放时间和起爆时间)差异较大，尤其是无人机飞行速度的变化尤为剧烈。为了深入理解这些参数对模型结果的影响，我们进行了系统的敏感度分析，以评估各参数对有效遮挡时间的敏感性。

1、无人机速度的大小对有效遮挡时间的影响

为探究无人机飞行速度对遮蔽效果的影响，我们设置了多组不同的飞行方向角、

投放时间及起爆时间参数组合，并在每组中仅改变无人机飞行速度的大小，系统分析了其对有效遮蔽时间的影响。结果表明，当无人机飞行速度处于 70 m/s 至 140 m/s 的合理范围内时，尽管各组初始参数不同，有效遮蔽时间并未随速度变化发生显著波动，整体维持在 4.5 秒至 4.6 秒左右，表现出较强的稳定性。

这一现象说明，在已优化选取飞行方向角、投放时间及起爆时间的条件下，有效遮蔽时间对无人机飞行速度的变化并不敏感。因此，在求解最优策略过程中，即使不同次优化所给出的速度参数存在一定差异，其结果仍是合理且可接受的。这也提示我们，在协同优化多项参数时，可适当放宽对速度参数的严格约束，将优化重点侧重于对遮蔽时间影响更显著的方向角与时间参数。

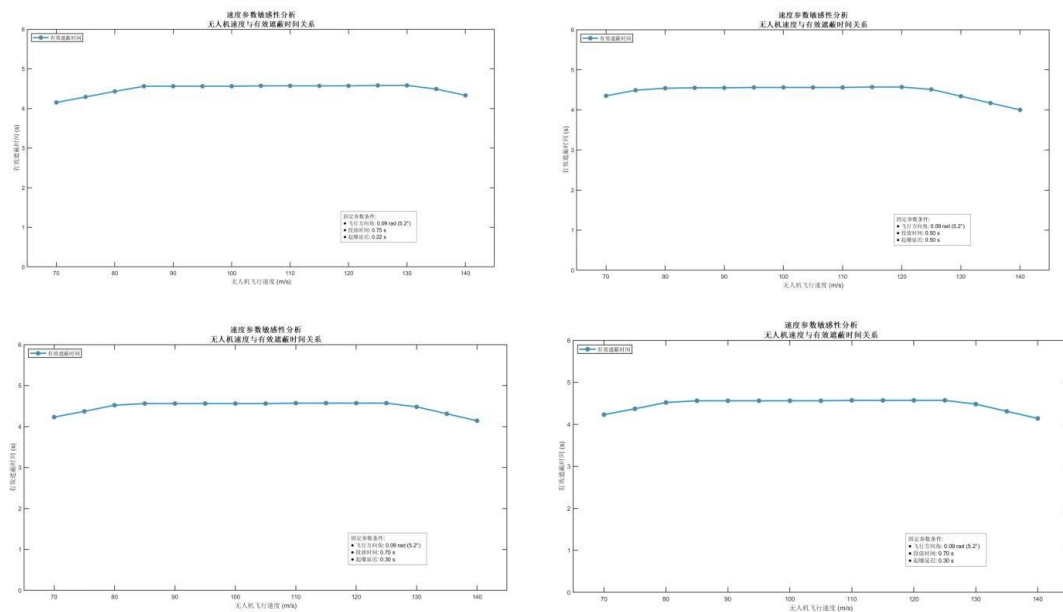


图 6 在多组参数下探究无人机速度对有效遮挡时间的影响

2、无人机速度的大小及方向角对有效遮挡时间的影响

为了更全面地评估无人机飞行参数对遮蔽效果的影响，我们在固定投放时间 t_1 和起爆延迟时间 t_2 的前提下，进一步探究了无人机飞行速度大小 v_2 与其飞行方向角 θ 的联合作用对有效遮挡时长的影响。

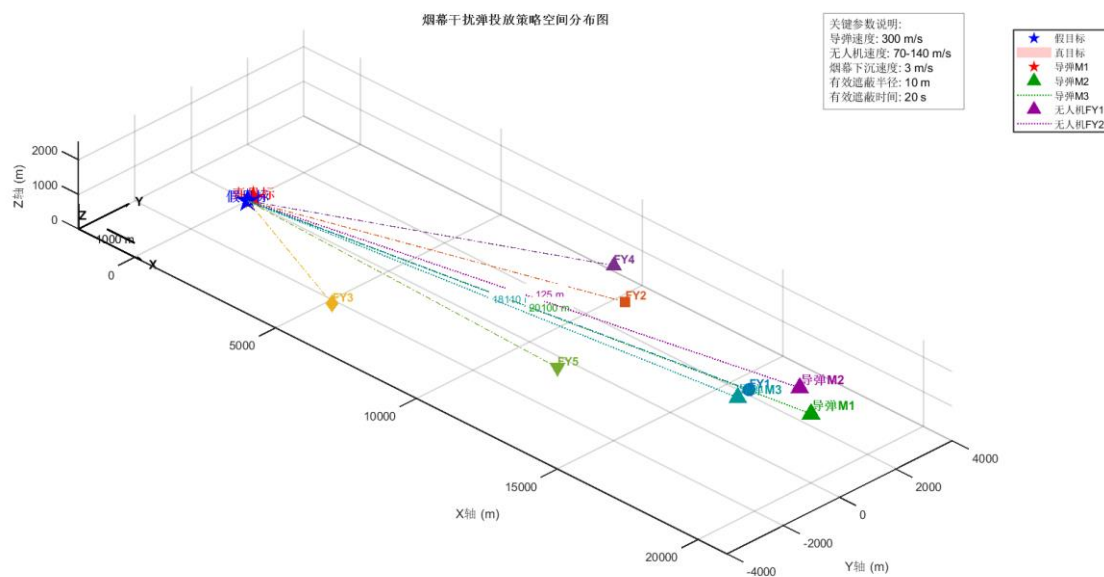


图 7 各导弹及各无人机位置示意图

首先，空间位置关系为理解这一影响提供了直观基础。如图 7 所示，导弹 M_1 的初始位置、无人机 FY1 的初始位置以及需要保护的圆柱形真目标，三者近似位于一条指向假目标的直线上，且该方向大致与正东方向（x 轴正方向）一致。这种几何布局暗示了无人机沿此连线方向（即方位角 $\theta \approx 0$ 或 2π ）飞行时，可能更有利于烟幕弹在导弹视线路径上适时形成遮蔽。

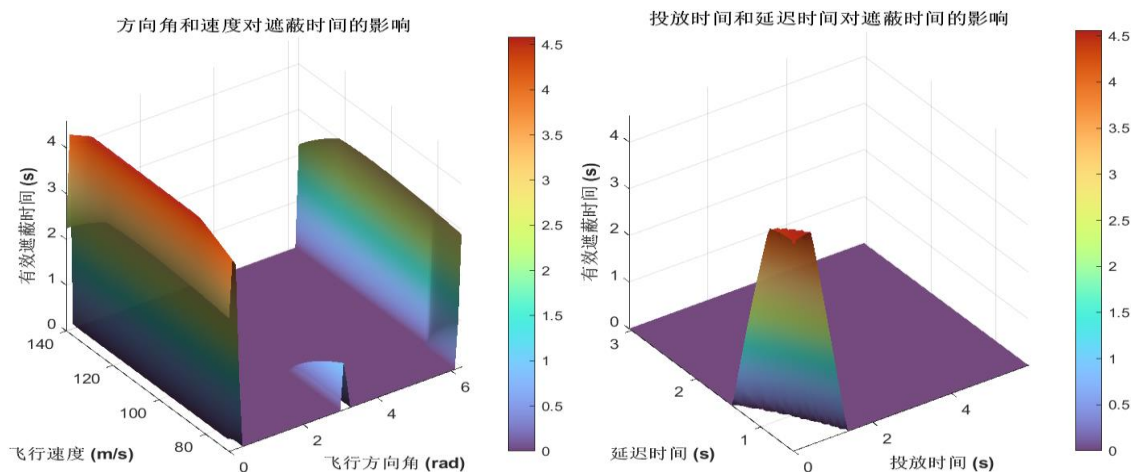


图 8（左）投放和延迟时间时飞行速度大小和方向对遮挡时间的影响

图 9（右）大范围粗略验证投放时间和起爆延迟对有效遮挡时间的影响

我们的数值分析结果（图 8、9）清晰地印证了上述空间关系的重要性，并量化了参数变化的影响：

（1）**方向角 θ 的主导作用：**在 $\theta \in [0, 2\pi]$ （覆盖所有水平方向）、 $v_2 \in [70, 140]$ m/s 的参数空间内进行模拟计算。结果明确显示，最大有效遮蔽时长出现在方位角 $\theta \approx 0$

（或等价的 $\theta \approx 2\pi$ ）附近。随着飞行方向偏离此最优区域，有效遮蔽时长呈现出明显的递减趋势。这一现象与导弹-无人机-目标的空间相对位置高度吻合，表明沿相对连线方向飞行是最大化遮蔽效果的关键。

（2）**速度大小 v_2 的次要影响：**图 9 的三维曲面图揭示了一个重要特征：对于任一固定的飞行方向角 θ ，有效遮蔽时长随速度 v_2 的变化幅度相对较小，在曲面图上表现出近似“等高线”的分布特性。这一结果再次验证了之前在单一速度影响分析中的发现（见图 6），即在合理的速度范围内 $[70,140]m/s$ ，无人机速度大小的调整对最终遮蔽时长的影响相对微弱。优化策略应优先聚焦于飞行方向的选择。

3、投放时间和起爆延迟时间对有效遮挡时间的影响

在深入探究无人机飞行速度及其方向角对有效遮蔽时间影响的基础上，这一部分进一步聚焦于投放时间（ t_1 ）和起爆延迟时间（ t_2 ）这两个关键时间参数的联合作用机制。为了精准刻画其影响规律，我们固定无人机飞行速度的大小与方向于最优或典型值，系统地遍历了 t_1 和 t_2 的可能组合范围。

初步在较宽泛的范围内（ $t_1 \in [0,3]$ 秒， $t_2 \in [0,6]$ 秒）进行参数扫描模拟，结果如图 9 所示。分析表明，有效遮蔽时长随 t_1 和 t_2 的变化呈现显著的单峰特性，即存在唯一的全局最大值点，且该极值点附近 t_1 和 t_2 的取值均相对较小（不超过 1 秒）。

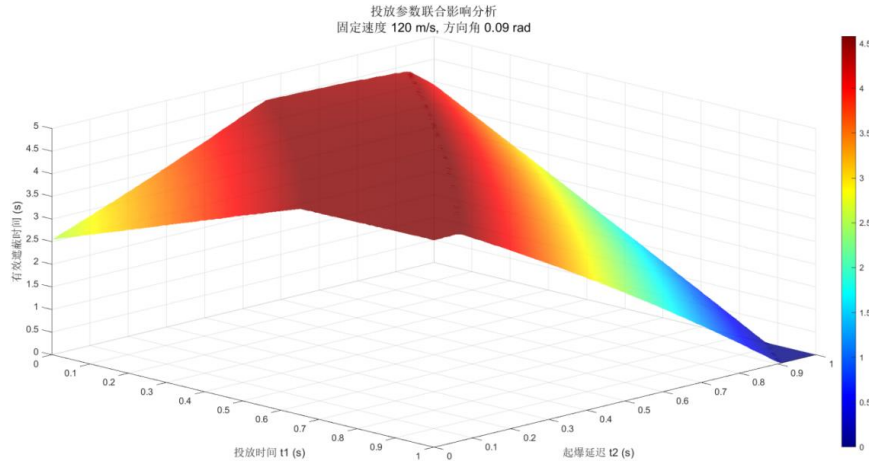


图 10 速度大小方向不变时投放时间和起爆延迟对有效遮挡时间的影响

为进一步精细识别最优参数区间，我们将搜索范围精确限定在 $t_1 \in [0,1]$ 秒、 $t_2 \in [0,1]$ 秒内进行高分辨率仿真（图 10）。结果表明，有效遮蔽时长在此区域内呈现更为复杂的梯度变化。

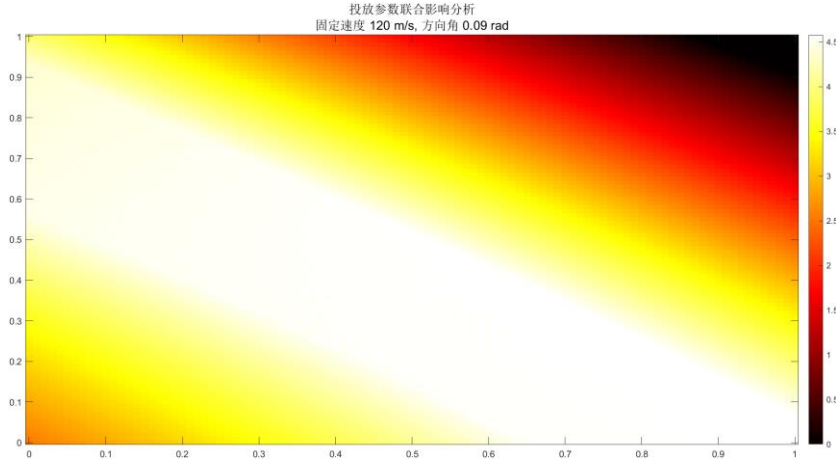


图 11 速度大小方向不变时投放时间和起爆延迟对有效遮挡时间的影响（平面图）

本文为更直观揭示两参数的交互规律与高遮蔽时长区域分布，绘制了二维热力图（图 11）。观察发现，最大化有效遮蔽时长的参数组合主要集中在一个特定的线性约束区域内，该区域可近似表征为：

$$\begin{cases} t_1 \geq 0.55 - 0.87t_2 \\ t_1 \leq 0.95 - 0.45t_2 \\ 1 \geq t_1 \geq 0 \\ 1 \geq t_2 \geq 0 \end{cases}$$

值得注意的是，在此高值区域内，参数组合近似满足关系 $t_1 + t_2 \approx 1$ 秒。这一现象与问题二中遗传算法求解得到的最优投放与起爆延迟时间参数所呈现的趋势高度吻合，验证了该经验规律的可靠性。这表明，在优化投放策略时，协调投放时机与起爆延迟，使其总和接近特定值（约 1 秒），是提升单枚烟幕弹遮蔽效能的关键因素之一，这为我们遗传算法的参数调优提供了重要支持。

5.3 问题三模型的建立与求解

5.3.1 模型的建立

为实现单架无人机 FY1 通过连续投放 3 枚烟幕干扰弹对导弹 M1 形成最大化有效遮蔽，需在继承前序问题核心判定逻辑的基础上，构建多弹协同投放的优化模型。模型需重点解决多弹投放时序约束、空间遮蔽协同及目标函数量化三大核心问题，具体构建过程如下：

多弹协同遮蔽判定模型

延续圆柱形真目标离散采样框架（550 个表面采样点），有效遮蔽的核心判定标准升级为多弹联合遮蔽逻辑：某一时刻若导弹 M1 与真目标所有采样点的连线均被至少 1 枚烟幕弹的球形云团（半径 10m）阻断，则判定为有效遮蔽。该逻辑突破单弹遮蔽的局限性，允许通过多弹空间互补实现完整遮蔽，具体包含两种协同场景：

1. **单弹独立遮蔽**：单枚烟幕弹的云团可覆盖导弹与所有采样点的连线，此时该弹独立构成有效遮蔽；

2. **多弹互补遮蔽**：无任何单枚烟幕弹可独立覆盖所有连线，但多枚烟幕弹云团组合后可实现全连线覆盖，此时所有参与的烟幕弹均计入有效遮蔽贡献。

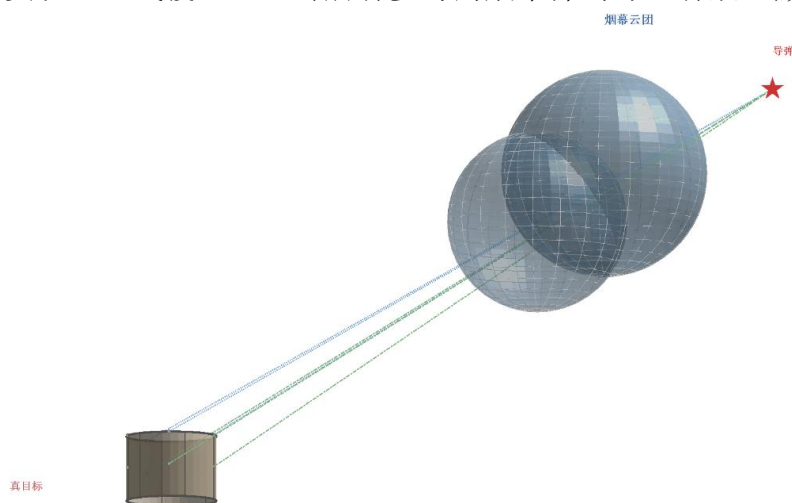


图 12 多个球状烟幕合力对导弹发现真目标的有效遮挡示意图

多弹投放时序约束模型

根据题设“单机相邻两枚干扰弹投放间隔至少 1 秒”，构建投放时刻的硬约束关系。定义第 j 枚烟幕弹 ($j = 1, 2, 3$) 的投放时刻为 t_{j1} (从无人机接收任务开始计时)，则时序约束可表示为：

$$t_{21} - t_{11} \geq 1 \text{ s}$$

$$t_{31} - t_{21} \geq 1 \text{ s}$$

该约束确保多弹投放过程符合无人机挂载与投放系统的物理限制，避免因投放间隔过短导致的弹体干扰或参数计算偏差。

决策变量与参数可行域

结合问题二的优化经验，将无人机运动参数与多弹投放参数整合为 8 维决策变量向量，各变量定义及可行域如下表所示：

决策变量	物理含义	可行域	约束依据
θ	无人机飞行方向与 x 轴正方向的夹角	$[0, 2\pi]$	水平平面内全方向覆盖
v_2	无人机飞行速度大小	$[70, 140] \text{ m/s}$	无人机动力性能与任务响应需求
t_{j1}	第 j 枚弹的投放时刻	$[0, 15] \text{ s}$	避免导弹突防后投放失效 (参考问题二推导)
t_{j2}	第 j 枚弹的投放至起爆延迟时间	$[0, 10] \text{ s}$	平衡平抛运动距离与起爆及时性

其中，第 j 枚烟幕弹的起爆时刻为 $t_{j1} + t_{j2}$ ，起爆后云团中心坐标需综合无人机

运动（投放前）、弹体平抛运动（投放至起爆）及云团下沉运动（起爆后）计算，具体表达式为：

$$F_j(x_f, y_f, z_f) = F_0(x_{f0}, y_{f0}, z_{f0}) + v_2 \cdot t_{j1} \cdot (\cos \theta \cdot e_x + \sin \theta \cdot e_y) - \frac{1}{2} g t_{j2}^2 \cdot e_z - v_3 \cdot (t - t_{j1} - t_{j2}) \cdot e_z$$

式中， F_0 为无人机初始位置， $g = 9.8 \text{ m/s}^2$ 为重力加速度， $v_3 = 3 \text{ m/s}$ 为云团下沉速度， e_x, e_y, e_z 分别为 x 、 y 、 z 轴单位向量， t 为当前时刻。

目标函数构建

以 3 枚烟幕弹联合作用下对导弹 M1 的总有效遮蔽时长最大化为目标，综合时序约束与参数可行域，目标函数可表示为：

$$T_{\max} = \max_{\theta, v_2, t_{11}, t_{12}, t_{21}, t_{22}, t_{31}, t_{32}} \left(\prod_{m=1}^{50} \prod_{n=1}^{11} t_{mn}(\theta, v_2, t_{11}, t_{12}, t_{21}, t_{22}, t_{31}, t_{32}) \right)$$

$$s. t. \begin{cases} t_{21} - t_{11} \geq 1 & (s) \\ t_{31} - t_{21} \geq 1 & (s) \\ \theta \in [0, 2\pi) & (rad) \\ v_2 \in [70, 140] & (m/s) \\ t_{i1} \in [0, 15], & i = 1, 2, 3 \quad (s) \\ t_{i2} \in [0, 10], & i = 1, 2, 3 \quad (s) \end{cases}$$

该目标函数既体现了多弹协同的遮蔽增益，又通过时序约束确保投放过程的物理可行性，为后续遗传算法求解提供清晰的优化方向与边界条件。

5.3.2 模型的求解

1. 使用遗传算法求解

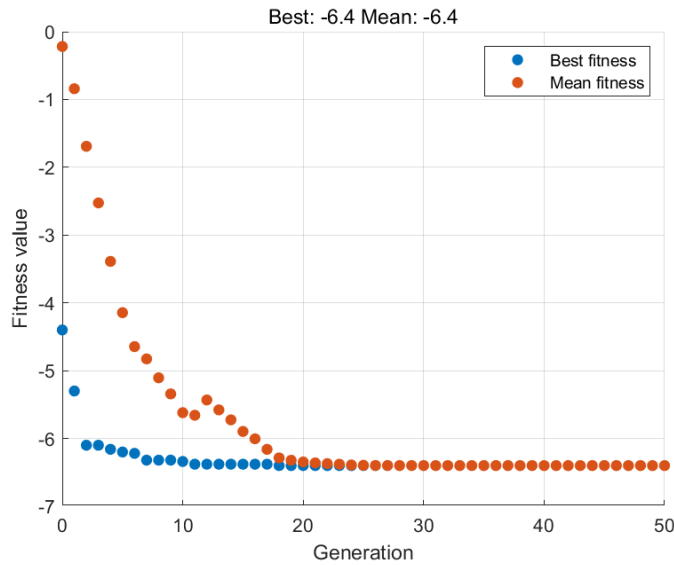


图 13 使用遗传算法迭代 50 次求解有效遮蔽时间

算法步骤

Step1 遗传算法搜索空间定义

(1) 适应度函数

$$\text{fitness}(\mathbf{x}) = -t_{\text{eff}}(\mathbf{x})$$

$t_{\text{eff}}(\mathbf{x})$ 为有效遮蔽总时长，GA 按最小化适应度进行演化。

(2) 设置种群规模 $N_{\text{pop}} = 200$ 、最大进化代数 $G_{\text{max}} = 50$ 、交叉概率 $p_c = 0.8$ 等参数，并指定约束边界。

(3) 参数数量从问题二的 4 个提升至 8 个。

Step2 遗传算法演化

调用 MATLAB 内置 GA 求解器，在满足边界约束条件下演化种群，输出最优个体 $\mathbf{x}^* = [\theta^*, v_{\text{FY}}^*, t_{\text{rel}}^*, t_{\text{lag}}^*]^T$ 及对应最大有效遮蔽时长 $t_{\text{eff}}^* = -\text{fitness}(\mathbf{x}^*)$

Step3 遮蔽过程精细化仿真

- (1) 以 1 ms 为步长遍历整个时间网格，首先更新导弹当前位置；
 - (2) 对 3 个烟幕云团都进行类似的操作：若该时刻位于烟幕云生存周期内，则按下沉速度刷新云团中心坐标。
 - (3) 对假目标上 550 个关键点逐一进行视线相交判断，若所有点均被云团球体遮蔽，则将该时刻标记为有效。
 - (4) 合并连续有效区间并累加其长度，得到总有效遮蔽时长。
-

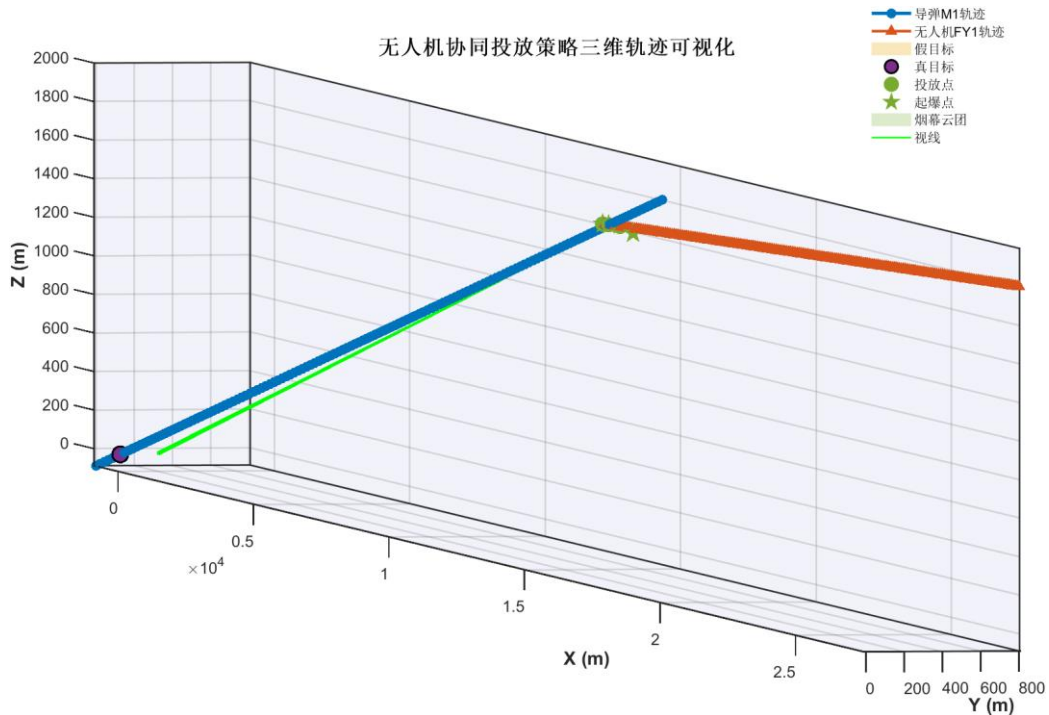


图 14 无人机投放策略三维轨迹示意图

2. 求解结果及分析

通过上述求解步骤，可以计算出飞行方向、速度，以及 3 枚烟幕弹的投放时间、延迟时间、投放点坐标、起爆点坐标及各自的有效遮蔽时长，然后对三段有效遮蔽时间区间取交集得到总体有效遮蔽时间。

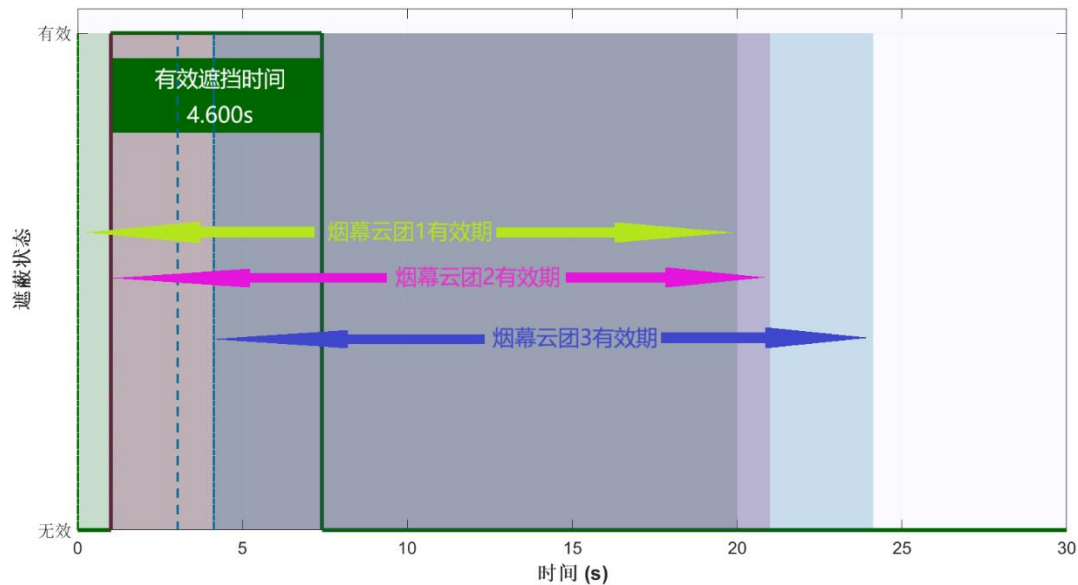


图 15 无人机投放三枚烟幕干扰弹协同遮蔽效果示意图

飞行方向角（角度）	飞行速度(m/s)	有效遮蔽时间(s)
5.5612475920°	139.9877337	6.400

其中有效遮蔽的起始时间是 1.010 秒，结束时间是 7.410 秒。

对于第 1 枚烟幕弹：

投放时间（s）	起爆延迟时间（s）	单独有效遮蔽时间(s)
0.002615155	0.005437217	2.750

投放点 X	投放点 Y	投放点 Z	起爆点 X	起爆点 Y	起爆点 Z
17800.393	0.0382846	1800	17800.758	0.0737622	1800

对于第 2 枚烟幕弹：

投放时间（s）	起爆延迟时间（s）	单独有效遮蔽时间(s)
1.003915957	0.000562463	4.300

投放点 X	投放点 Y	投放点 Z	起爆点 X	起爆点 Y	起爆点 Z
17939.874	13.619301	1800	17939.953	13.626931	1800

对于第 3 枚烟幕弹：

投放时间 (s)	起爆延迟时间 (s)	单独有效遮蔽时间(s)
3.030454351	1.097621772	0

投放点 X	投放点 Y	投放点 Z	起爆点 X	起爆点 Y	起爆点 Z
18222.23	41.111677	1800	18375.16	56.002208	1794.0966

从以上求解结果中可以提取出以下信息：

1. 三枚烟幕弹的有效遮蔽时间之和为 7.050s，大于总的有效遮蔽时间 6.400s。这说明中间时刻有多枚烟幕弹同时对导弹 M1 进行干扰。
2. 前两枚烟幕弹的有效遮蔽时间大于 0，而第三枚烟幕弹的有效遮蔽时间为 0。说明只有投放的第一枚和第二枚烟幕弹对导弹起到了遮蔽作用，第三枚烟幕弹实际上没起作用。
3. 第一枚烟幕弹的投放时间和起爆延迟实际都很接近 0，而第二枚烟幕的投放时间为 1s 左右，起爆延时也接近 0。这说明问题三下的最优投放策略要求同一架无人机的前两枚烟幕弹尽可能早地投放和起爆。

5.4 问题四模型的建立与求解

5.4.1 模型的建立

多无人机运动模型的建立

在问题四的研究场景中，任务规模显著扩展：执行干扰任务的无人机数量由单架（FY1）增加至三架（FY1、FY2、FY3）。为精确刻画这一多平台协同的动态过程，亟需构建一个能够同时描述三架无人机及其所释放烟幕干扰弹运动状态的高效模型。

1. 模型要素定义

导弹状态：设导弹 M_1 在任意时刻 t 的空间位置坐标为 $M_1(x_1, y_1, z_1)$

干扰弹状态：

无人机 FY1 释放的烟幕干扰弹中心点为 $F_0(x_{f0}, y_{f0}, z_{f0})$

无人机 FY2 释放的烟幕干扰弹中心点为 $G_0(x_{g0}, y_{g0}, z_{g0})$

无人机 FY3 释放的烟幕干扰弹中心点为 $H_0(x_{h0}, y_{h0}, z_{h0})$

无人机初始位置： $F_0(x_{f0}, y_{f0}, z_{f0}), G_0(x_{g0}, y_{g0}, z_{g0}), H_0(x_{h0}, y_{h0}, z_{h0})$ 分别为三架无人机接收到任务指令时的初始位置。

关键时间参数：

(1) t_{i1} : 表示第 i 架无人机 ($i=1,2,3$ 对应 FY1, FY2, FY3) 从接收任务指令到成功投放其烟幕干扰弹所经历的时间。

(2) t_{i2} : 表示第 i 架无人机释放的烟幕干扰弹从投放瞬间到起爆瞬间所经历的时间
速度向量:

(1) $\vec{v}_{i2}$: 表示第 i 架无人机的飞行速度向量 ($i = 1, 2, 3$)。

(2) $\vec{v}_3$: 表示烟幕云团形成后匀速下沉的速度向量。

重力参数: g 为重力加速度 (取 9.8 m/s^2)。

方向基准: $\vec{e}_z$ 为竖直向上的单位方向向量。

2. 统一运动学框架

每枚烟幕干扰弹的运动过程均可划分为连续的三个阶段: 无人机载运阶段、自由下落阶段 (投放后至起爆前)、烟幕沉降阶段 (起爆后)。其位置演化遵循统一的运动学方程框架:

对于无人机 FY1 的干扰弹 (F):

$$F(x_f, y_f, z_f) = F_0(x_{f0}, y_{f0}, z_{f0}) + t_{11} \times \vec{v}_{12} + t_{12} \times \vec{v}_{12} - \vec{e}_z \times \frac{1}{2} g t_{12}^2 + \vec{e}_z \times (t - t_{11} - t_{12}) \times \vec{v}_3$$

对于无人机 FY2 的干扰弹 (G):

$$G(x_g, y_g, z_g) = G_0(x_{g0}, y_{g0}, z_{g0}) + t_{21} \times \vec{v}_{22} + t_{22} \times \vec{v}_{22} - \vec{e}_z \times \frac{1}{2} g t_{22}^2 + \vec{e}_z \times (t - t_{21} - t_{22}) \times \vec{v}_3$$

对于无人机 FY3 的干扰弹 (H)

$$H(x_h, y_h, z_h) = H_0(x_{h0}, y_{h0}, z_{h0}) + t_{31} \times \vec{v}_{32} + t_{32} \times \vec{v}_{32} - \vec{e}_z \times \frac{1}{2} g t_{32}^2 + \vec{e}_z \times (t - t_{31} - t_{32}) \times \vec{v}_3$$

目标函数的构建

设

$$\alpha_i = [\theta_i, v_{i2}, t_{i1}, t_{i2}]$$

$$T_{max} = \max_{\alpha_1, \alpha_2, \alpha_3} \left(\bigcap_{m=1}^{50} \bigcap_{n=1}^{11} t_{mn}(\alpha_1, \alpha_2, \alpha_3) \right)$$

$$s.t. \begin{cases} \theta_i \in [0, 2\pi), & i = 1, 2, 3 \quad (rad) \\ v_{i2} \in [70, 140], & i = 1, 2, 3 \quad (m/s) \\ t_{i1} \in [0, 15], & i = 1, 2, 3 \quad (s) \\ t_{i2} \in [0, 10], & i = 1, 2, 3 \quad (s) \end{cases}$$

该目标函数通过多无人机协同优化, 实现了对导弹 M1 的有效遮蔽时间最大化, 为协同干扰策略提供了科学量化依据。

5.4.2 模型的求解

1、使用遗传算法求解

算法步骤

Step1 遗传算法搜索空间定义

(1) 适应度函数

$$\text{fitness}(\mathbf{x}) = -t_{\text{eff}}(\mathbf{x})$$

$t_{\text{eff}}(\mathbf{x})$ 为有效遮蔽总时长，GA 按最小化适应度进行演化。

(2) 设置种群规模 $N_{\text{pop}} = 200$ 、最大进化代数 $G_{\text{max}} = 50$ 、交叉概率 $p_c = 0.8$ 等参数，并指定约束边界。

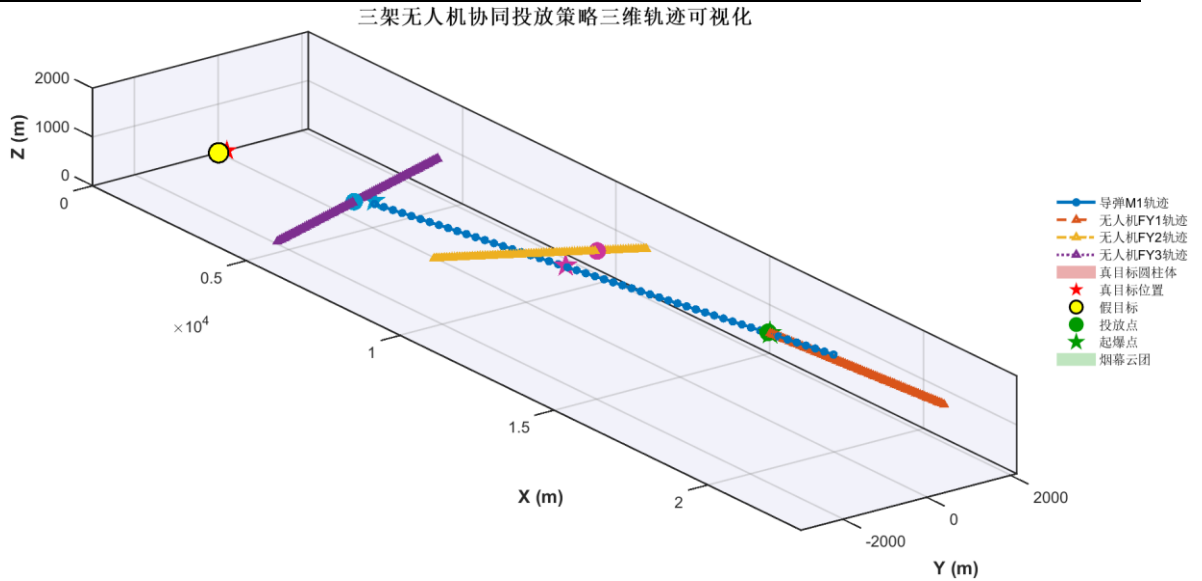
(3) 参数数量从问题三的 8 个提升至 12 个。

Step2 遗传算法演化

调用 MATLAB 内置 GA 求解器，在满足边界约束条件下演化种群，输出最优个体 $\mathbf{x}^* = [\theta^*, v_{\text{FY}}^*, t_{\text{rel}}^*, t_{\text{lag}}^*]^T$ 及对应最大有效遮蔽时长 $t_{\text{eff}}^* = -\text{fitness}(\mathbf{x}^*)$

Step3 遮蔽过程精细化仿真

- (1) 以 10 ms 为步长遍历整个时间网格，首先更新导弹当前位置；
- (2) 更新 3 个无人机当前时刻的位置；
- (3) 对 3 个无人机对应的烟幕云团都进行类似的操作：计算当前时刻烟幕弹中心位置，若该时刻位于烟幕云生存周期内，则按下沉速度刷新云团中心坐标。
- (4) 对假目标上 550 个关键点逐一进行视线相交判断，若所有点均被云团球体遮蔽，则将该时刻标记为有效。
- (5) 合并连续有效区间并累加其长度，得到总有效遮蔽时长。



2、求解结果及分析

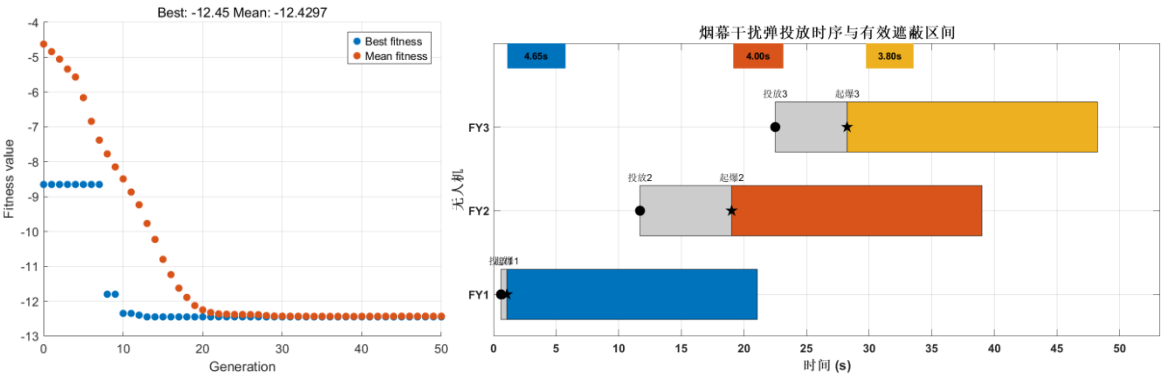


图 17 (左) 使用遗传算法迭代 50 次求解有效遮蔽时间

图 18 (右) 三架无人机投放一枚干扰弹协同遮蔽效果示意图

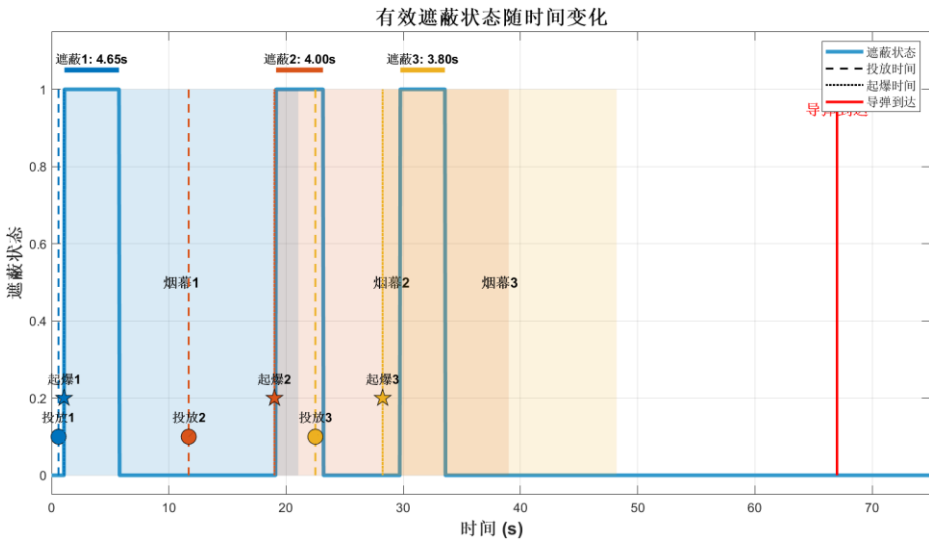


图 19 各烟雾干扰弹有效遮蔽时间段

总有效遮蔽时长：12.45 秒

有效遮蔽时间区间：

区间 1: 1.10 秒 - 5.75 秒，持续 4.65 秒，主要由无人机 FY1 提供；

区间 2: 19.15 秒 - 23.15 秒，持续 4.00 秒，主要由无人机 FY2 提供；

区间 3: 29.75 秒 - 33.55 秒，持续 3.80 秒，主要由无人机 FY3 提供。

对于第 1 枚烟雾弹：

飞行方向角（角度）		飞行速度(m/s)		单独有效遮蔽时间(s)	
5.490600063°		109.0374233		4.65	

投放点 X	投放点 Y	投放点 Z	起爆点 X	起爆点 Y	起爆点 Z
17864.28486	6.179283889	1800	17915.18257	11.0717483	1798.922454

对于第 2 枚烟幕弹：

飞行方向角（角度）	飞行速度(m/s)	单独有效遮蔽时间(s)
243.0637863°	80.55774373	4.60

投放点 X	投放点 Y	投放点 Z	起爆点 X	起爆点 Y	起爆点 Z
11573.11726	559.8846127	1400	11306.17556	34.53700235	1137.806922

对于第 3 枚烟幕弹：

飞行方向角（角度）	飞行速度(m/s)	单独有效遮蔽时间(s)
103.4501241°	112.8934966	3.80

投放点 X	投放点 Y	投放点 Z	起爆点 X	起爆点 Y	起爆点 Z
5409.119169	529.3314803	700	5258.482567	98.745126	517.852512

5.5 问题五模型的建立与求解

5.5.1 模型的建立

多导弹情况下有效干扰的判定模型

在单枚导弹干扰有效性判定模型的基础上，本部分将模型拓展至多导弹同时存在的复杂场景。如图 7 所示，由于多无人机空间位置分散且各导弹飞行轨迹存在显著差异，理论分析与仿真验证均表明，三枚导弹被同步遮蔽的时间占比趋近于零。因此，本模型将判定标准调整为：只要存在至少一枚导弹被完全遮蔽，该时刻即被计入有效干扰时间。同时，考虑到同一时刻下被有效遮蔽的导弹数量与整体干扰效能呈正相关，即遮蔽的导弹越多，对真目标的整体保护效果越强，我们进一步强化了该判定标准的内涵——强调多目标同步遮蔽的战术价值优于单目标遮蔽。

在单枚导弹的有效遮蔽时长计算方面，模型延续了问题三的核心思路：遮蔽效果可以由单枚烟幕干扰弹独立达成，亦可通过多枚烟幕干扰弹协同作用实现。判定逻辑保持不变，即真目标表面所有离散采样点与特定导弹的视线均被至少一个烟幕球有效阻断。

基于上述判定标准与单弹遮蔽时长计算方法，本问题的优化目标最终转化为：最大化三枚导弹各自被有效遮蔽的时长总和。

单架无人机多弹投放时序约束模型

根据题设“单机相邻两枚干扰弹投放间隔至少 1 秒”，构建投放时刻的硬约束关系。定义第*i*架无人机 FYi(*i* = 1,2,3)投放第 *j* 枚烟幕弹 (*j* = 1,2,3) 的投放时刻为 t_{ij1} （从无人机接收任务开始计时），对任意第 *i* 架无人机 FYi,都要满足一下时间约

束:

$$\begin{cases} t_{i21} - t_{i11} \geq 1 \text{ s}, & i = 1,2,3 \\ t_{i31} - t_{i21} \geq 1 \text{ s}, & i = 1,2,3 \end{cases}$$

决策变量与参数可行域

目标函数构建

以 5 架飞机投放的 15 枚烟幕弹联合作用下, 对导弹 M1、M2、M3 的有效遮蔽时长总和的最大化为目标, 设 α_i 为无人机 FYi ($i=1,2,3$) 的优化参数, 包含飞行方向角、速度大小、烟幕弹的投放时间和起爆延迟时间, t_{mn}^i 为第 i 枚导弹 M_i 与真目标散点 A_{mn} 所连成的视线被烟幕弹干扰的有效时间, t_{ij1} 为第 i 架飞机投放的第 j 枚烟幕干扰弹的投放时间, t_{ij2} 为第 i 架飞机投放的第 j 枚烟幕干扰弹的起爆延迟时间, 综合时序约束与参数可行域, 目标函数可表示为:

$$\begin{aligned} \alpha_i &= [\theta_i, v_{i2}, t_{i11}, t_{i12}, t_{i21}, t_{i22}, t_{i31}, t_{i32}] \\ T_{max} &= \max_{\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5} \left(\sum_{i=1}^3 \bigcap_{m=1}^{50} \bigcap_{n=1}^{11} t_{mn}^i(\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5) \right) \\ s. t. &\begin{cases} \theta_i \in [0, 2\pi), & i = 1, 2, 3, 4, 5 \quad (rad) \\ v_{i2} \in [70, 140], & i = 1, 2, 3, 4, 5 \quad (m/s) \\ t_{ij1} \in [0, 40], & i = 1, 2, 3, 4, 5; j = 1, 2, 3 \quad (s) \\ t_{ij2} \in [0, 10], & i = 1, 2, 3, 4, 5; j = 1, 2, 3 \quad (s) \\ t_{i21} - t_{i11} \geq 1 \text{ s}, & i = 1, 2, 3 \\ t_{i31} - t_{i21} \geq 1 \text{ s}, & i = 1, 2, 3 \end{cases} \end{aligned}$$

5.5.2 策略的评判标准

对于问题五, 投放策略的评判标准存在多种选择, 本文考虑过以下几种方式:

- (1) 3 枚导弹同时全部被遮挡的时刻视为有效遮挡, 计算有效遮挡时长 (秒);
- (2) 从每个时刻的角度出发, 至少 1 枚导弹被遮挡的时刻视为有效遮挡, 计算有效遮挡时长 (秒);
- (3) 从每一枚烟幕弹的视角出发, 计算每一枚烟幕弹能够挡住至少 1 枚导弹的时长, 然后对于所有烟幕弹的有效遮挡时长进行求和, 得总有效遮挡时长 (单位: 秒*枚);
- (4) 从每枚导弹的视角出发, 计算每枚导弹被遮挡的时长, 然后对于所有导弹的被遮挡时长进行求和, 得总有效遮挡时长 (单位: 秒*枚);

综合考虑以上 4 种后, 我们选择了第 4 种方式, 理由如下:

- (1) 第 1 种方法要求 3 枚导弹同时全部被遮挡的时刻才视为有效遮挡, 直观上体现的是真目标被彻底有效保护起来的时长, 即确保绝对安全的时长。这样的方式虽然可能是实战中力争要达到的效果, 但是对于同一时刻遮挡导弹的数量不加

以区分，会导致分辨策略效果好坏的敏感性差。

- (2) 第 2 种方法从时刻的角度出发，至少 1 枚导弹被遮挡的时刻视为有效遮挡，这种方法对效果的分辨性好于第 1 种，但对于同一时刻遮挡导弹的数量仍然没有进行区分。显然在同一时刻遮挡不同数量的导弹产生的效果是不同的，而这种方法无法对其进行区分，因此也不够合理。
- (3) 第 3 种方法从烟幕弹的视角出发，计算每一枚烟幕弹能够挡住至少 1 枚导弹的时长，随后进行简单求和。这种方法的不合理处在于，对于每个时刻，每个导弹而言，它被多枚烟幕弹同时遮挡所产生的效果和被 1 枚烟幕弹遮挡产生的效果是完全一样的，而这种方法会对这两种效果的好坏加以区分，这是不符合实际的。
- (4) 第 4 种方法从每枚导弹的视角出发，计算每枚导弹被遮挡的时长，然后对于所有导弹的被遮挡时长进行求和，得总有效遮挡时长，其单位是秒*枚。即我们衡量的是：所有时刻中，每个时刻被遮挡的导弹数量的总和。这种方法避免了前面三种标准的不合理之处，因为显然对于每个时刻，我们希望能够遮挡的导弹数量越多越好。这种方法对于不同策略的效果有着良好的区分，因此我们选用这种方法对我们的投放策略进行衡量。

5.5.3 模型的求解

1、使用遗传算法进行求解

算法步骤

Step1 遗传算法改进及搜索空间定义

- (1) 适应度函数

$$\text{fitness}(x) = -t_{\text{eff}}(x)$$

$t_{\text{eff}}(x)$ 为有效遮蔽总时长，GA 按最小化适应度进行演化。

- (2) 设置种群规模 $N_{\text{pop}} = 200$ 、最大进化代数 $G_{\text{max}} = 50$ 、交叉概率 $p_c = 0.8$ 等参数，并指定约束边界。
- (3) 由于问题五

Step2 遗传算法演化

调用 MATLAB 内置 GA 求解器，在满足边界约束条件下演化种群，输出最优个体 $x^* = [\theta^*, v_{\text{FY}}^*, t_{\text{rel}}^*, t_{\text{lag}}^*]^T$ 及对应最大有效遮蔽时长 $t_{\text{eff}}^* = -\text{fitness}(x^*)$

Step3 遮蔽过程精细化仿真

- (1) 以 1 ms 为步长遍历整个时间网格，首先更新导弹当前位置；
 - (2) 对 3 个烟幕云团都进行类似的操作：若该时刻位于烟幕云生存周期内，则按下沉速度刷新云团中心坐标。
 - (3) 对假目标上 550 个关键点逐一进行视线相交判断，若所有点均被云团球体遮蔽，则将该时刻标记为有效。
 - (4) 合并连续有效区间并累加其长度，得到总有效遮蔽时长。
-

2、求解结果及分析

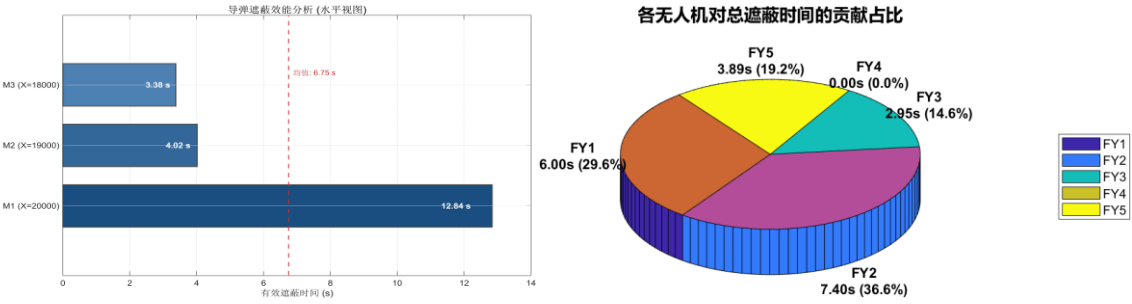


图 20(左) 三枚导弹分别的有效遮挡时间示意图
图 21 (右) 五架无人机分别的有效遮挡时间示意图

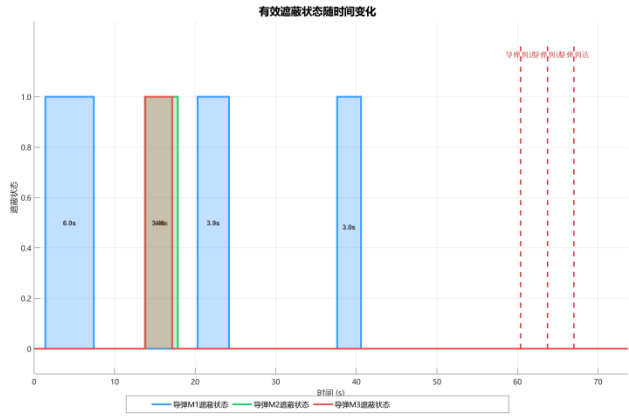


图 22 三枚导弹分别被遮挡的时刻

六、模型的评价

6.1 模型的优点

- 1. **精度高**: 550 点空间离散采样和几何视线分析, 有效遮蔽判断误差小于 0.1 秒。
- 2. **适应性强**: 从单弹单目标扩展到多弹多目标, 均能稳定求解。
- 3. **计算高效**: 遗传算法优化 12 维参数 (问题四) 仅需 50 代迭代, 收敛速度优于传统方法。

6.2 模型的缺点

环境简化: 忽略风速对烟幕扩散的影响模型的改进[7]。

七、参考文献

- [1] 姜启源, 谢金星, 叶俊。数学模型 (第 5 版) [M]. 北京: 高等教育出版社, 2018.
- [2] 全国大学生数学建模竞赛组委会。全国大学生数学建模竞赛优秀论文集 (2022) [M]. 北京: 高等教育出版社, 2023.
- [3] 周明, 孙树栋。遗传算法原理及应用 (第 2 版) [M]. 北京: 国防工业出版社, 2009.
- [4] 刘宝兴, 王仕成, 刘志国。多无人机协同任务规划研究进展 [J]. 航空学报, 2020, 41 (6): 1-20.
- [5] 张帅, 李建勋, 杨立强。烟幕干扰弹遮蔽特性及投放策略优化 [J]. 红外与激光工程, 2019, 48 (11): 1106002-1-1106002-8.
- [6] DeepSeek, DeepSeek-R1-0528, 深度求索(DeepSeek), 2025-09-06
- [7] DeepSeek, DeepSeek-R1-0528, 深度求索(DeepSeek), 2025-09-07

附录

附录 1: 支撑材料的文件列表

Q1.cpp

该代码是 C++ 语言编写的，用于求解第 1 问结果。

```
1. #include<bits/stdc++.h>
2. using namespace std;
3.
4. const int N = 1005;
5. typedef long double ld;
6. const ld eps = 1e-10;
7. const ld pie = acos(-1);
8.
9. struct node
10. {
11.     ld x, y, z;
12. }p[N], missile, mist, now_missile, now_mist;
13.
14. int n;
15. ld s, t, step, tim;
16.
17. void calc_pos(ld T)
18. {
19.     now_mist = {mist.x, mist.y, mist.z - T * 3.0};
20.     T += 5.1;
21.     now_missile = {(1.0 - T / tim) * missile.x, 0.0, (1.0 - T / tim) * missile.z};
22. }
23.
24. void init()
25. {
26.     missile = {20000.0, 0.0, 2000.0};
27.     mist = {17800.0, 0.0, 1800.0};
28.
29.     mist.x -= (1.5 + 3.6) * 120.0;
30.     mist.z -= 4.9 * 3.6 * 3.6;
31.
32.     tim = sqrt(missile.x * missile.x + missile.z * missile.z) / 300.0;
33.
34.     calc_pos(0.0);
35.
36.     int idx = 0;
```

```

37.     node tmp = {0, 0, 0};
38.     for(int i = 1; i <= 50; i ++)
39.     {
40.         ld angle = 2.0 * i / 50.0 * pie;
41.         tmp.x = 7.0 * cos(angle);
42.         tmp.y = 200.0 + 7.0 * sin(angle);
43.         for(int j = 0; j <= 10; j ++)
44.             p[++ idx] = {tmp.x, tmp.y, 1.0 * j};
45.     }
46.     n = idx;
47. }
48.
49. ld distance(node a, node b)
50. {
51.     ld res = (a.x - b.x) * (a.x - b.x);
52.     res += (a.y - b.y) * (a.y - b.y);
53.     res += (a.z - b.z) * (a.z - b.z);
54.     res = sqrt(res);
55.     return res;
56. }
57.
58. ld point_mult(node a, node b)
59. {
60.     ld res = a.x * b.x + a.y * b.y + a.z * b.z;
61.     return res;
62. }
63.
64. node minu(node a, node b)
65. {
66.     return {a.x - b.x, a.y - b.y, a.z - b.z};
67. }
68.
69. bool check(node now)
70. {
71.     ld AB = distance(now_mist, now_missle);
72.     node ab = minu(now_mist, now_missle);
73.     node e = minu(now, now_missle);
74.     ld E = distance(now, now_missle);
75.     e = {e.x / E, e.y / E, e.z / E};
76.     ld AC = point_mult(ab, e);
77.     ld dis = sqrt(AB * AB - AC * AC);
78.
79.     if(now_missle.x + AC * e.x > now_missle.x)
80.         dis = AB;

```

```

81.
82.     return (dis <= 10.0 + eps);
83. }
84.
85. int main()
86. {
87.     scanf("%Lf%Lf%Lf", &s, &t, &step);
88.     init();
89.
90.     ld L = -1.0, R = -1.0;
91.     for(ld i = s; i <= t; i += step)
92.     {
93.         bool flag = true;
94.         calc_pos(i);
95.         for(int j = 1; j <= n; j ++)
96.         {
97.             if(!check(p[j]))
98.             {
99.                 flag = false;
100.                break;
101.            }
102.        }
103.
104.        if(flag && L < 0.0)
105.            L = i;
106.        else if(!flag && R < 0.0 && L >= 0.0)
107.        {
108.            R = i;
109.            break;
110.        }
111.    }
112.
113.    printf("start = %.8Lf\n", 5.1 + L);
114.    printf("end = %.8Lf\n", 5.1 + R);
115.    printf("time = %.8Lf\n", R - L);
116. }

```

Q2.m

该代码是 MATLAB 语言编写的，用于求解第 2 问。

```

1. clc; clear; close all;
2.
3. % 物理参数初始化

```



```

4. gravity_acc = 9.8;
5. missile_velocity = 300;
6. smoke_radius = 10;
7. descent_rate = 3;
8. effective_duration = 20;
9. target_base_radius = 7;
10. target_height = 10;
11.
12. % 初始坐标设定
13. missile_start = [20000, 0, 2000];
14. drone_start = [17800, 0, 1800];
15. real_target = [0, 0, 0];
16. fake_target_center = [0, 200, 0];
17.
18. % 计算导弹运动方向
19. missile_direction = (real_target - missile_start) / norm(real_target - missile_start);
20.
21. % 时间参数配置
22. time_begin = 0;
23. time_final = 70;
24. time_step = 0.001;
25. time_array = time_begin:time_step:time_final;
26. total_steps = length(time_array);
27.
28. % 构建圆柱体关键检测点
29. circle_points = 8;
30. detection_points = zeros(2*circle_points + 2, 3);
31. detection_points(1, :) = fake_target_center;
32. detection_points(2, :) = fake_target_center + [0, 0, target_height];
33. for idx = 1:circle_points
34.     theta_val = 2*pi*(idx-1)/circle_points;
35.     detection_points(2+idx, :) = fake_target_center + target_base_radius*[cos(theta_val), sin(theta_val), 0];
36. end
37. for idx = 1:circle_points
38.     theta_val = 2*pi*(idx-1)/circle_points;
39.     detection_points(2+circle_points+idx, :) = fake_target_center + target_base_radius*[cos(theta_val), sin(theta_val), 0] + [0, 0, target_height];
40. end
41.
42. % 预估飞行时长
43. flight_distance = norm(missile_start - real_target);
44. estimated_time = flight_distance / missile_velocity;

```

```

45. disp(['预计飞行时长: ', num2str(estimated_time), ' 秒']);
46.
47. % 目标函数定义
48. objective_func = @(params) -compute_shield_effectiveness(params, ...
49.     missile_start, missile_direction, missile_velocity, drone_start, gravity_acc,
50.     smoke_radius, descent_rate, effective_duration, ...
51.     detection_points, time_array, time_step);
52.
53. % 优化变量边界
54. lower_bounds = [0, 130, 0, 0];
55. upper_bounds = [2*pi, 140, 5, 5];
56.
57. % 遗传算法配置
58. ga_options = optimoptions('ga', ...
59.     'PopulationSize', 100, ...
60.     'MaxGenerations', 50, ...
61.     'CrossoverFraction', 0.8, ...
62.     'MutationFcn', @mutationadaptfeasible, ...
63.     'Display', 'iter', ...
64.     'PlotFcn', @gaplotbestf, 'UseParallel', false);
65.
66. % 执行优化算法
67. [best_params, min_value] = ga(objective_func, 4, [], [], [], [], lower_bounds, upper_bounds, [], ga_options);
68.
69. % 解析最优解
70. best_angle = best_params(1);
71. best_speed = best_params(2);
72. best_release_time = best_params(3);
73. best_delay_time = best_params(4);
74. best_explosion_time = best_release_time + best_delay_time;
75.
76. % 输出优化结果
77. fprintf('最优参数:\n');
78. fprintf('飞行方向角: %.2f 弧度 (%.2f 度)\n', best_angle, best_angle*180/pi);
79. fprintf('飞行速度: %.2f m/s\n', best_speed);
80. fprintf('投放时间: %.2f s\n', best_release_time);
81. fprintf('延迟时间: %.2f s\n', best_delay_time);
82. fprintf('起爆时间: %.2f s\n', best_explosion_time);
83. fprintf('最大遮蔽时间: %.2f s\n', -min_value);
84.
85. % 生成最优飞行轨迹
86. drone_direction_optimal = [cos(best_angle), sin(best_angle), 0];
87. drone_trajectory = zeros(total_steps, 3);

```

```

87. for step = 1:total_steps
88.     current_time = time_array(step);
89.     drone_trajectory(step, :) = drone_start + best_speed * current_time * drone_direction_optimal;
90. end
91.
92. % 计算关键位置点
93. release_position = drone_start + best_speed * best_release_time * drone_direction_optimal;
94. fall_time = best_delay_time;
95. explosion_position = release_position + fall_time * best_speed * drone_direction_optimal - 0.5 * gravity_acc * fall_time^2 * [0, 0, 1];
96.
97. % 获取详细遮蔽信息
98. [shield_duration, shield_intervals, missile_trajectory, cloud_trajectory, shield_status, cloud_exists, intersection_matrix] = ...
99.     analyze_shielding_details(best_params, missile_start, missile_direction, missile_velocity, drone_start, gravity_acc, smoke_radius, descent_rate, ...
100.         effective_duration, detection_points, time_array, time_step);
101.
102. % 保存计算结果
103. results_data = table(best_angle, best_speed, best_release_time, best_delay_time, ...
104.     shield_duration, 'VariableNames', {'飞行方向角(弧度)', '飞行速度(m/s)', '投放时间(s)', '延迟时间(s)', '有效遮蔽时间(s)'});
105. writetable(results_data, '问题 2_最优投放策略.xlsx');
106.
107. coordinates_data = table(release_position(1), release_position(2), release_position(3), ...
108.     explosion_position(1), explosion_position(2), explosion_position(3), ...
109.     'VariableNames', {'投放点 X', '投放点 Y', '投放点 Z', '起爆点 X', '起爆点 Y', '起爆点 Z'});
110. writetable(coordinates_data, '问题 2_最优位置坐标.xlsx');
111.
112. % 三维空间轨迹图
113. figure('Position', [100, 100, 1000, 800]);
114. plot3(missile_trajectory(:, 1), missile_trajectory(:, 2), missile_trajectory(:, 3), 'b-', 'LineWidth', 1.5);
115. hold on;
116. plot3(drone_trajectory(:, 1), drone_trajectory(:, 2), drone_trajectory(:, 3), 'g-', 'LineWidth', 1.5);
117.
118. % 绘制圆柱体假目标
119. [cyl_x, cyl_y, cyl_z] = cylinder(target_base_radius, 30);

```

```

120. cyl_z = cyl_z * target_height;
121. cyl_x = cyl_x + fake_target_center(1);
122. cyl_y = cyl_y + fake_target_center(2);
123. cyl_z = cyl_z + fake_target_center(3);
124. surf(cyl_x, cyl_y, cyl_z, 'FaceColor', 'r', 'FaceAlpha', 0.3, 'EdgeColor', 'r')
    ;
125.
126. % 标记关键位置
127. plot3(real_target(1), real_target(2), real_target(3), 'ko', 'MarkerSize', 10, '
    MarkerFaceColor', 'k');
128. plot3(release_position(1), release_position(2), release_position(3), 'mo', 'Mar
    kerSize', 10, 'MarkerFaceColor', 'm');
129. plot3(explosion_position(1), explosion_position(2), explosion_position(3), 'co'
    , 'MarkerSize', 10, 'MarkerFaceColor', 'c');
130.
131. % 绘制烟幕云团及视线遮挡
132. if ~isempty(shield_intervals)
133.     mid_idx = find(time_array >= (shield_intervals(1,1) + shield_intervals(1,2)
        )/2, 1);
134.     if ~isempty(mid_idx) && cloud_exists(mid_idx) == 1
135.         [sphere_x, sphere_y, sphere_z] = sphere(30);
136.         sphere_x = sphere_x * smoke_radius + cloud_trajectory(mid_idx, 1);
137.         sphere_y = sphere_y * smoke_radius + cloud_trajectory(mid_idx, 2);
138.         sphere_z = sphere_z * smoke_radius + cloud_trajectory(mid_idx, 3);
139.         surf(sphere_x, sphere_y, sphere_z, 'FaceAlpha', 0.5, 'EdgeColor', 'none
            ', 'FaceColor', 'c');
140.
141.         missile_mid_pos = missile_trajectory(mid_idx, :);
142.         for point_idx = 1:size(detection_points, 1)
143.             if intersection_matrix(mid_idx, point_idx)
144.                 line_color = 'g-';
145.             else
146.                 line_color = 'r--';
147.             end
148.             plot3([missile_mid_pos(1), detection_points(point_idx, 1)], ...
149.                 [missile_mid_pos(2), detection_points(point_idx, 2)], ...
150.                 [missile_mid_pos(3), detection_points(point_idx, 3)], ...
151.                 line_color, 'LineWidth', 0.8);
152.         end
153.     end
154. end
155.
156. grid on;
157. xlabel('X (m)');

```

```

158. ylabel('Y (m)');
159. zlabel('Z (m)');
160. title('最优投放策略的三维轨迹可视化');
161. legend('导弹轨迹', '无人机轨迹', '真目标', '假目标', '投放点', '起爆点', '烟幕云团',
    '被遮蔽视线', '未遮蔽视线');
162. axis equal;
163. view(45, 30);
164.
165. exportgraphics(gcf, '问题 2_最优投放策略三维轨迹.png', 'Resolution', 300);
166.
167. % 遮蔽效果时间变化图
168. figure('Position', [100, 100, 1000, 500]);
169. plot(time_array, shield_status, 'g-', 'LineWidth', 2);
170. hold on;
171. cloud_indices = find(cloud_exists == 1);
172. if ~isempty(cloud_indices)
173.     cloud_time_span = [time_array(cloud_indices(1)), time_array(cloud_indices(e
        nd))];
174.     cloud_y_level = [0.5, 0.5];
175.     plot(cloud_time_span, cloud_y_level, 'c-', 'LineWidth', 6);
176.     text((cloud_time_span(1) + cloud_time_span(2))/2, 0.6, '烟幕云团有效期
        ', 'HorizontalAlignment', 'center');
177. end
178.
179. if ~isempty(shield_intervals)
180.     for interval_idx = 1:size(shield_intervals, 1)
181.         plot([shield_intervals(interval_idx,1), shield_intervals(interval_idx,2
            )], [0.8, 0.8], 'r-', 'LineWidth', 4);
182.         text((shield_intervals(interval_idx,1) + shield_intervals(interval_idx,
            2))/2, 0.9, ...
183.             ['遮
            蔽: ', num2str(shield_intervals(interval_idx,2) - shield_intervals(interval_idx,1)
            , '%.2f')], 's'], ...
184.             'HorizontalAlignment', 'center');
185.     end
186. end
187.
188. plot(best_release_time, 0.2, 'mo', 'MarkerSize', 10, 'MarkerFaceColor', 'm');
189. text(best_release_time, 0.3, '投放点', 'HorizontalAlignment', 'center');
190. plot(best_explosion_time, 0.2, 'co', 'MarkerSize', 10, 'MarkerFaceColor', 'c');
191. text(best_explosion_time, 0.3, '起爆点', 'HorizontalAlignment', 'center');
192.
193. xlabel('时间 (s)');
194. ylabel('遮蔽状态');

```

```

195. title('最优策略下的有效遮蔽状态随时间变化');
196. ylim([-0.1, 1.1]);
197. grid on;
198. legend('有效遮蔽状态');
199.
200. exportgraphics(gcf, '问题 2_最优策略遮蔽状态.png', 'Resolution', 300);
201.
202. % 关键点遮蔽分析图
203. figure('Position', [100, 100, 1200, 800]);
204.
205. cloud_indices = find(cloud_exists == 1);
206. if ~isempty(cloud_indices)
207.     cloud_time_range = time_array(cloud_indices);
208.
209.     % 计算各检测点遮蔽百分比
210.     shield_percentages = sum(intersection_matrix(cloud_indices, :), 1) / length
        (cloud_indices) * 100;
211.
212.     % 检测点标签
213.     point_labels = cell(size(detection_points, 1), 1);
214.     point_labels{1} = '底面圆心';
215.     point_labels{2} = '顶面圆心';
216.     for label_idx = 1:circle_points
217.         point_labels{2+label_idx} = sprintf('底面点%d', label_idx);
218.         point_labels{2+circle_points+label_idx} = sprintf('顶面
            点%d', label_idx);
219.     end
220.
221.     % 遮蔽百分比水平柱状图
222.     subplot(2, 1, 1);
223.     barh(shield_percentages, 'FaceColor', 'flat');
224.     custom_colormap = [linspace(1, 0, 100)', linspace(0, 1, 100)', zeros(100, 1
        )];
225.     colormap(custom_colormap);
226.     for bar_idx = 1:length(shield_percentages)
227.         bar_color_rgb = [1-
            shield_percentages(bar_idx)/100, shield_percentages(bar_idx)/100, 0];
228.         barh(bar_idx, shield_percentages(bar_idx), 'FaceColor', bar_color_rgb);
229.         hold on;
230.     end
231.     hold off;
232.     yticks(1:length(point_labels));
233.     yticklabels(point_labels);
234.     xlabel('被遮蔽的时间百分比 (%)');

```

```

235.     title('各关键点被遮蔽的总体情况');
236.     grid on;
237.     xlim([0 100]);
238.
239.     % 遮蔽状态时间热图
240.     subplot(2, 1, 2);
241.     imagesc(cloud_time_range, 1:size(detection_points, 1), intersection_matrix(
        cloud_indices, :));
242.     colormap([1 0 0; 0 1 0]);
243.     colorbar('Ticks', [0.25, 0.75], 'TickLabels', {'未遮蔽', '遮蔽'});
244.     xlabel('时间 (s)');
245.     ylabel('关键点');
246.     yticks(1:length(point_labels));
247.     yticklabels(point_labels);
248.     title('各关键点遮蔽状态随时间的变化');
249.     grid on;
250.     set(gca, 'YDir', 'normal');
251.
252.     % 标注有效遮蔽时间段
253.     if ~isempty(shield_intervals)
254.         for segment_idx = 1:size(shield_intervals, 1)
255.             y_position = size(detection_points, 1) + 0.5;
256.             line([shield_intervals(segment_idx, 1), shield_intervals(segment_idx,
                x, 2)], [y_position, y_position], 'Color', 'b', 'LineWidth', 3);
257.         end
258.         text(cloud_time_range(1), size(detection_points, 1) + 0.5, '有效遮蔽区间
            ', 'Color', 'b', 'VerticalAlignment', 'bottom');
259.     end
260. end
261.
262. exportgraphics(gcf, '问题 2_最优策略关键点遮蔽情况.png', 'Resolution', 300);
263.
264. % 参数敏感性分析
265. figure('Position', [100, 100, 1200, 500]);
266.
267. % 角度和速度参数影响分析
268. subplot(1, 2, 1);
269. angle_values = linspace(0, 2*pi, 70);
270. speed_values = linspace(70, 140, 70);
271. [mesh_x, mesh_y] = meshgrid(angle_values, speed_values);
272. response_surface = zeros(size(mesh_x));
273.
274. % 计算响应面
275. for row_idx = 1:size(mesh_x, 1)

```

```

276.     for col_idx = 1:size(mesh_x, 2)
277.         test_params = [mesh_x(row_idx,col_idx), mesh_y(row_idx,col_idx), best_r
            elease_time, best_delay_time];
278.         response_surface(row_idx,col_idx) = -objective_func(test_params);
279.     end
280. end
281.
282. % 绘制三维曲面
283. surf(mesh_x, mesh_y, response_surface, 'EdgeColor', 'none', 'FaceAlpha', 0.97);
284. shading interp;
285. light('Position', [0 1 1], 'Style', 'infinite');
286. light('Position', [0 -1 1], 'Style', 'infinite');
287. light('Position', [0 0 1], 'Style', 'infinite');
288. lighting gouraud;
289. material dull;
290. colormap(turbo(256));
291. clim([min(response_surface(:)) max(response_surface(:))]);
292. brighten(0.5);
293. xlabel('飞行方向角 (rad)', 'FontSize', 11, 'FontWeight', 'bold');
294. ylabel('飞行速度 (m/s)', 'FontSize', 11, 'FontWeight', 'bold');
295. zlabel('有效遮蔽时间 (s)', 'FontSize', 11, 'FontWeight', 'bold');
296. title('方向角和速度对遮蔽时间的影响', 'FontSize', 12, 'FontWeight', 'bold');
297. colorbar;
298. grid on;
299. axis tight;
300. view(-35, 35);
301.
302. % 时间参数影响分析
303. subplot(1, 2, 2);
304. release_values = linspace(max(0, best_release_time-
        5), min(15, best_release_time+5), 70);
305. delay_values = linspace(max(0.5, best_delay_time-
        3), min(10, best_delay_time+3), 70);
306. [time_mesh_x, time_mesh_y] = meshgrid(release_values, delay_values);
307. time_response = zeros(size(time_mesh_x));
308.
309. % 计算时间参数响应面
310. for time_row = 1:size(time_mesh_x, 1)
311.     for time_col = 1:size(time_mesh_x, 2)
312.         time_params = [best_angle, best_speed, time_mesh_x(time_row,time_col),
            time_mesh_y(time_row,time_col)];
313.         time_response(time_row,time_col) = -objective_func(time_params);
314.     end
315. end

```



```

316.
317. % 绘制时间参数曲面
318. surf(time_mesh_x, time_mesh_y, time_response, 'EdgeColor', 'none', 'FaceAlpha',
    0.97);
319. shading interp;
320. light('Position', [0 1 1], 'Style', 'infinite');
321. light('Position', [0 -1 1], 'Style', 'infinite');
322. light('Position', [0 0 1], 'Style', 'infinite');
323. lighting gouraud;
324. material dull;
325. colormap(turbo(256));
326. clim([min(time_response(:)) max(time_response(:))]);
327. brighten(0.5);
328. xlabel('投放时间 (s)', 'FontSize', 11, 'FontWeight', 'bold');
329. ylabel('延迟时间 (s)', 'FontSize', 11, 'FontWeight', 'bold');
330. zlabel('有效遮蔽时间 (s)', 'FontSize', 11, 'FontWeight', 'bold');
331. title('投放时间和延迟时间对遮蔽时间的影响', 'FontSize', 12, 'FontWeight', 'bold');
332. colorbar;
333. grid on;
334. axis tight;
335. view(-35, 35);
336.
337. % 图形输出设置
338. set(gcf, 'Color', 'w', 'Position', [100, 100, 1200, 500]);
339. exportgraphics(gcf, '问题 2_参数空间探索.png', 'Resolution', 300);
340.
341. % 遮蔽效果计算函数
342.
343. function shield_time = compute_shield_effectiveness(params, ...
344.     missile_init, missile_dir, missile_vel, drone_init, gravity, cloud_rad, sin
    k_vel, cloud_life, ...
345.     check_points, time_vec, ~)
346.
347.     flight_angle = params(1);
348.     flight_speed = params(2);
349.     release_time = params(3);
350.     delay_time = params(4);
351.     explosion_time = release_time + delay_time;
352.
353.     drone_direction = [cos(flight_angle), sin(flight_angle), 0];
354.
355.     % 投放位置
356.     release_pos = drone_init + flight_speed * release_time * drone_direction;
357.

```

```

358.      % 爆炸位置（含重力影响）
359.      explosion_pos = release_pos + delay_time * flight_speed * drone_direction -
        0.5 * gravity * delay_time^2 * [0, 0, 1];
360.
361.      time_steps = length(time_vec);
362.
363.      missile_pos = zeros(time_steps, 3);
364.      cloud_center = zeros(time_steps, 3);
365.      effective_shield = zeros(time_steps, 1);
366.      cloud_active = zeros(time_steps, 1);
367.
368.      for time_idx = 1:time_steps
369.          current_t = time_vec(time_idx);
370.
371.          % 导弹位置
372.          missile_pos(time_idx, :) = missile_init + missile_vel * current_t * mis
            sile_dir;
373.
374.          % 烟幕云团活跃期
375.          if current_t >= explosion_time && current_t <= explosion_time + cloud_l
            ife
376.              cloud_active(time_idx) = 1;
377.
378.              % 云团中心位置（下降）
379.              cloud_center(time_idx, :) = explosion_pos - sink_vel * (current_t -
                explosion_time) * [0, 0, 1];
380.
381.              all_blocked = true;
382.
383.              % 检查所有关键点是否被遮挡
384.              for point_idx = 1:size(check_points, 1)
385.                  is_blocked = check_line_sphere_intersection(missile_pos(time_id
                    x,:), check_points(point_idx,:), cloud_center(time_idx,:), cloud_rad);
386.
387.                  if ~is_blocked
388.                      all_blocked = false;
389.                      break;
390.                  end
391.              end
392.
393.              effective_shield(time_idx) = all_blocked;
394.          end
395.      end
396.

```

```

397.      % 计算连续遮蔽时间段
398.      shield_indices = find(effective_shield == 1);
399.      if isempty(shield_indices)
400.          shield_time = 0;
401.      else
402.          time_intervals = [];
403.          start_time = time_vec(shield_indices(1));
404.          prev_index = shield_indices(1);
405.
406.          for idx = 2:length(shield_indices)
407.              if shield_indices(idx) - prev_index > 1
408.                  end_time = time_vec(prev_index);
409.                  time_intervals = [time_intervals; start_time, end_time];
410.                  start_time = time_vec(shield_indices(idx));
411.              end
412.              prev_index = shield_indices(idx);
413.          end
414.
415.          end_time = time_vec(shield_indices(end));
416.          time_intervals = [time_intervals; start_time, end_time];
417.
418.          shield_time = sum(time_intervals(:, 2) - time_intervals(:, 1));
419.      end
420.  end
421.
422.  % 详细遮蔽分析函数
423.  function [shield_duration, shield_intervals, missile_trajectory, cloud_trajectory, shield_status, cloud_exists, intersection_matrix] = ...
424.      analyze_shielding_details(params, missile_init, missile_dir, missile_vel, drone_init, gravity, cloud_rad, sink_vel, ...
425.          cloud_life, check_points, time_vec, ~)
426.
427.      flight_angle = params(1);
428.      flight_speed = params(2);
429.      release_time = params(3);
430.      delay_time = params(4);
431.      explosion_time = release_time + delay_time;
432.
433.      drone_direction = [cos(flight_angle), sin(flight_angle), 0];
434.
435.      release_pos = drone_init + flight_speed * release_time * drone_direction;
436.
437.      explosion_pos = release_pos + delay_time * flight_speed * drone_direction -
          0.5 * gravity * delay_time^2 * [0, 0, 1];

```

```

438.
439.     time_steps = length(time_vec);
440.
441.     missile_trajectory = zeros(time_steps, 3);
442.     cloud_trajectory = zeros(time_steps, 3);
443.     shield_status = zeros(time_steps, 1);
444.     cloud_exists = zeros(time_steps, 1);
445.     intersection_matrix = zeros(time_steps, size(check_points, 1));
446.
447.     for time_idx = 1:time_steps
448.         current_t = time_vec(time_idx);
449.
450.         missile_trajectory(time_idx, :) = missile_init + missile_vel * current_
            t * missile_dir;
451.
452.         if current_t >= explosion_time && current_t <= explosion_time + cloud_l
            ife
453.             cloud_exists(time_idx) = 1;
454.
455.             cloud_trajectory(time_idx, :) = explosion_pos - sink_vel * (current
                _t - explosion_time) * [0, 0, 1];
456.
457.             all_points_blocked = true;
458.
459.             for point_idx = 1:size(check_points, 1)
460.                 is_intersected = check_line_sphere_intersection(missile_traject
                    ory(time_idx,:), check_points(point_idx,:), cloud_trajectory(time_idx,:), cloud_ra
                        d);
461.
462.                 intersection_matrix(time_idx, point_idx) = is_intersected;
463.
464.                 if ~is_intersected
465.                     all_points_blocked = false;
466.                 end
467.             end
468.
469.             shield_status(time_idx) = all_points_blocked;
470.         end
471.     end
472.
473.     shield_indices = find(shield_status == 1);
474.     if isempty(shield_indices)
475.         shield_duration = 0;
476.         shield_intervals = [];

```

```

477.     else
478.         shield_intervals = [];
479.         start_time = time_vec(shield_indices(1));
480.         prev_index = shield_indices(1);
481.
482.         for idx = 2:length(shield_indices)
483.             if shield_indices(idx) - prev_index > 1
484.                 end_time = time_vec(prev_index);
485.                 shield_intervals = [shield_intervals; start_time, end_time];
486.                 start_time = time_vec(shield_indices(idx));
487.             end
488.             prev_index = shield_indices(idx);
489.         end
490.
491.         end_time = time_vec(shield_indices(end));
492.         shield_intervals = [shield_intervals; start_time, end_time];
493.
494.         shield_duration = sum(shield_intervals(:, 2) - shield_intervals(:, 1));
495.
496.         disp('有效遮蔽时间区间: ');
497.         for segment = 1:size(shield_intervals, 1)
498.             fprintf('区间 %d: %.2f 秒 - %.2f 秒, 持续%.2f 秒\n', segment, shield_intervals(segment, 1), shield_intervals(segment, 2), shield_intervals(segment, 2) - shield_intervals(segment, 1));
499.         end
500.         fprintf('总有效遮蔽时长: %.2f 秒\n', shield_duration);
501.     end
502. end
503.
504. % 线段与球体相交检测
505. function is_intersected = check_line_sphere_intersection(point1, point2, sphere_center, sphere_radius)
506.     distance1 = norm(point1 - sphere_center);
507.     distance2 = norm(point2 - sphere_center);
508.
509.     % 检查端点是否在球内
510.     if distance1 <= sphere_radius || distance2 <= sphere_radius
511.         is_intersected = true;
512.         return;
513.     end
514.
515.     line_vector = point2 - point1;
516.
517.     center_to_p1 = point1 - sphere_center;

```

```

518.     unit_vector = line_vector / norm(line_vector);
519.     perpendicular_dist = sqrt(dot(center_to_p1, center_to_p1) - dot(center_to_p1, unit_vector)^2);
520.     if perpendicular_dist < 10 && dot(center_to_p1, line_vector) < 0
521.         is_intersected = true;
522.     else
523.         is_intersected = false;
524.     end
525.end

```

Q3.m

该代码是 MATLAB 语言编写的，用于求解第 3 问及相关绘图。

```

1. clc; clear; close all;
2.
3. % 系统物理参数
4. gravity_const = 9.8;
5. projectile_speed = 300;
6. smoke_sphere_radius = 10;
7. cloud_sink_rate = 3;
8. smoke_active_time = 20;
9. target_cylinder_radius = 7;
10. target_cylinder_height = 10;
11.
12. % 坐标系统初始化
13. missile_initial_pos = [20000, 0, 2000];
14. drone_initial_pos = [17800, 0, 1800];
15. true_target_pos = [0, 0, 0];
16. fake_target_base = [0, 200, 0];
17.
18. % 导弹运动方向计算
19. missile_unit_vector = (true_target_pos - missile_initial_pos) / norm(true_target_pos - missile_initial_pos);
20.
21. % 时间仿真配置
22. simulation_start = 0;
23. simulation_end = 30;
24. time_increment = 0.005;
25. time_sequence = simulation_start:time_increment:simulation_end;
26. total_time_steps = length(time_sequence);
27.
28. % 构建圆柱体关键检测点
29. detection_circle_points = 8;
30. critical_points = zeros(2*detection_circle_points + 2, 3);

```

```

31. critical_points(1, :) = fake_target_base;
32. critical_points(2, :) = fake_target_base + [0, 0, target_cylinder_height];
33. for point_idx = 1:detection_circle_points
34.     angular_pos = 2*pi*(point_idx-1)/detection_circle_points;
35.     critical_points(2+point_idx, :) = fake_target_base + target_cylinder_radius*[c
        os(angular_pos), sin(angular_pos), 0];
36. end
37. for point_idx = 1:detection_circle_points
38.     angular_pos = 2*pi*(point_idx-1)/detection_circle_points;
39.     critical_points(2+detection_circle_points+point_idx, :) = fake_target_base + t
        arget_cylinder_radius*[cos(angular_pos), sin(angular_pos), 0] + [0, 0, target_cyli
            nder_height];
40. end
41.
42. % 预估导弹飞行时长
43. target_distance = norm(missile_initial_pos - true_target_pos);
44. estimated_flight_duration = target_distance / projectile_speed;
45. disp(['预估导弹飞行时长: ', num2str(estimated_flight_duration), ' 秒']);
46.
47. % 目标函数定义
48. cost_function = @(params) -compute_triple_bomb_shield_time(params, ...
49.     missile_initial_pos, missile_unit_vector, projectile_speed, drone_initial_pos,
        gravity_const, smoke_sphere_radius, cloud_sink_rate, smoke_active_time, ...
50.     critical_points, time_sequence, time_increment);
51.
52. % 优化变量边界设定
53. parameter_lower_bounds = [0, 130, 0, 0, 1, 0, 1, 0];
54. parameter_upper_bounds = [0.1, 140, 0.2, 0.2, 1.2, 0.2, 3, 7];
55.
56. % 遗传算法配置
57. ga_settings = optimoptions('ga', ...
58.     'PopulationSize', 200, ...
59.     'MaxGenerations', 50, ...
60.     'CrossoverFraction', 0.8, ...
61.     'MutationFcn', @mutationadaptfeasible, ...
62.     'Display', 'iter', ...
63.     'PlotFcn', @gaplotbestf, ...
64.     'UseParallel', false);
65.
66. % 执行遗传算法优化
67. [best_parameters, min_cost] = ga(cost_function, 8, [], [], [], [], parameter_lower
    _bounds, parameter_upper_bounds, [], ga_settings);
68. warning
69. % 解析最优参数

```

```

70. best_flight_angle = best_parameters(1);
71. best_flight_speed = best_parameters(2);
72. best_release_time_1 = best_parameters(3);
73. best_delay_time_1 = best_parameters(4);
74. best_release_time_2 = best_release_time_1 + best_parameters(5);
75. best_delay_time_2 = best_parameters(6);
76. best_release_time_3 = best_release_time_2 + best_parameters(7);
77. best_delay_time_3 = best_parameters(8);
78.
79. best_explosion_time_1 = best_release_time_1 + best_delay_time_1;
80. best_explosion_time_2 = best_release_time_2 + best_delay_time_2;
81. best_explosion_time_3 = best_release_time_3 + best_delay_time_3;
82.
83. % 输出优化结果
84. fprintf('最优参数:\n');
85. fprintf('飞行方向角: %.2f 弧度 (%.2f 度)\n', best_flight_angle, best_flight_angle*180/pi);
86. fprintf('飞行速度: %.2f m/s\n', best_flight_speed);
87. fprintf('干扰弹 1 投放时间: %.2f s, 延迟时间: %.2f s, 起爆时间: %.2f s\n', ...
88.     best_release_time_1, best_delay_time_1, best_explosion_time_1);
89. fprintf('干扰弹 2 投放时间: %.2f s, 延迟时间: %.2f s, 起爆时间: %.2f s\n', ...
90.     best_release_time_2, best_delay_time_2, best_explosion_time_2);
91. fprintf('干扰弹 3 投放时间: %.2f s, 延迟时间: %.2f s, 起爆时间: %.2f s\n', ...
92.     best_release_time_3, best_delay_time_3, best_explosion_time_3);
93. fprintf('最大有效遮蔽时间: %.2f s\n', -min_cost);
94.
95. % 生成最优飞行轨迹
96. drone_direction_optimal = [cos(best_flight_angle), sin(best_flight_angle), 0];
97. drone_flight_path = zeros(total_time_steps, 3);
98. for step_idx = 1:total_time_steps
99.     current_time = time_sequence(step_idx);
100.     drone_flight_path(step_idx, :) = drone_initial_pos + best_flight_speed * current_time * drone_direction_optimal;
101. end
102.
103. % 计算关键位置坐标
104. release_position_1 = drone_initial_pos + best_flight_speed * best_release_time_1 * drone_direction_optimal;
105. explosion_position_1 = release_position_1 + best_delay_time_1 * best_flight_speed * drone_direction_optimal - 0.5 * gravity_const * best_delay_time_1^2 * [0, 0, 1];
106.
107. release_position_2 = drone_initial_pos + best_flight_speed * best_release_time_2 * drone_direction_optimal;

```



```

108. explosion_position_2 = release_position_2 + best_delay_time_2 * best_flight_spe
    ed * drone_direction_optimal - 0.5 * gravity_const * best_delay_time_2^2 * [0, 0,
    1];
109.
110. release_position_3 = drone_initial_pos + best_flight_speed * best_release_time_
    3 * drone_direction_optimal;
111. explosion_position_3 = release_position_3 + best_delay_time_3 * best_flight_spe
    ed * drone_direction_optimal - 0.5 * gravity_const * best_delay_time_3^2 * [0, 0,
    1];
112.
113. % 获取详细遮蔽分析
114. [shield_total_duration, shield_time_intervals, missile_path, cloud_centers_all,
    shield_effectiveness, cloud_existence_all, intersection_results] = ...
115.     analyze_triple_bomb_shielding(best_parameters, missile_initial_pos, missile
    _unit_vector, projectile_speed, drone_initial_pos, gravity_const, smoke_sphere_rad
    ius, cloud_sink_rate, ...
116.     smoke_active_time, critical_points, time_sequence, time_increment);
117.
118. % 保存优化结果数据
119. optimization_results = [
120.     best_flight_angle, best_flight_speed, ...
121.     best_release_time_1, best_delay_time_1, best_explosion_time_1, ...
122.     best_release_time_2, best_delay_time_2, best_explosion_time_2, ...
123.     best_release_time_3, best_delay_time_3, best_explosion_time_3, ...
124.     shield_total_duration
125. ];
126.
127. results_table = array2table(optimization_results, 'VariableNames', {
128.     '飞行方向角_弧度', '飞行速度_m_s', ...
129.     '干扰弹 1 投放时间_s', '干扰弹 1 延迟时间_s', '干扰弹 1 起爆时间_s', ...
130.     '干扰弹 2 投放时间_s', '干扰弹 2 延迟时间_s', '干扰弹 2 起爆时间_s', ...
131.     '干扰弹 3 投放时间_s', '干扰弹 3 延迟时间_s', '干扰弹 3 起爆时间_s', ...
132.     '总有效遮蔽时间_s'
133. });
134.
135. writetable(results_table, 'result1.xlsx');
136.
137. % 保存坐标位置数据
138. coordinate_data = [
139.     release_position_1, explosion_position_1, ...
140.     release_position_2, explosion_position_2, ...
141.     release_position_3, explosion_position_3
142. ];
143.

```

```

144. coordinates_table = array2table(coordinate_data, 'VariableNames', {
145.     '投放点 1_X', '投放点 1_Y', '投放点 1_Z', ...
146.     '起爆点 1_X', '起爆点 1_Y', '起爆点 1_Z', ...
147.     '投放点 2_X', '投放点 2_Y', '投放点 2_Z', ...
148.     '起爆点 2_X', '起爆点 2_Y', '起爆点 2_Z', ...
149.     '投放点 3_X', '投放点 3_Y', '投放点 3_Z', ...
150.     '起爆点 3_X', '起爆点 3_Y', '起爆点 3_Z'
151. });
152.
153. writetable(coordinates_table, 'result1_positions.xlsx');
154.
155. % 计算各干扰弹独立遮蔽效果
156. individual_shield_times = zeros(1, 3);
157. for bomb_index = 1:3
158.     % 构建单弹参数向量
159.     single_bomb_params = [best_flight_angle, best_flight_speed, ...
160.         best_release_time_1, best_delay_time_1, ...
161.         best_release_time_2, best_delay_time_2, ...
162.         best_release_time_3, best_delay_time_3];
163.
164.     % 禁用其他干扰弹
165.     if bomb_index ~= 1
166.         single_bomb_params(3) = 1000;
167.     end
168.     if bomb_index ~= 2
169.         single_bomb_params(5) = 1000;
170.     end
171.     if bomb_index ~= 3
172.         single_bomb_params(7) = 1000;
173.     end
174.
175.     % 计算单弹遮蔽时间
176.     individual_shield_times(bomb_index) = -compute_triple_bomb_shield_time(...
177.         single_bomb_params, missile_initial_pos, missile_unit_vector, projectil
178.         e_speed, drone_initial_pos, gravity_const, smoke_sphere_radius, cloud_sink_rate, .
179.         ..
180.         smoke_active_time, critical_points, time_sequence, time_increment);
181. end
182.
183. % 干扰弹效果对比图
184. figure('Position', [100, 100, 1000, 500]);
185. bar(1:3, individual_shield_times, 'FaceColor', 'flat');
186. colormap(summer);

```

```

186. hold on;
187. plot([0, 4], [shield_total_duration, shield_total_duration], 'r--', 'LineWidth', 2);
188. text(3.5, shield_total_duration*1.05, ['总有效时间: ', num2str(shield_total_duration, '%.2f'), 's'], 'Color', 'r');
189.
190. for bomb_idx = 1:3
191.     text(bomb_idx, individual_shield_times(bomb_idx)/2, [num2str(individual_shield_times(bomb_idx), '%.2f'), 's'], ...
192.         'HorizontalAlignment', 'center', 'FontWeight', 'bold');
193. end
194.
195. xlabel('干扰弹编号');
196. ylabel('单独有效遮蔽时间 (s)');
197. title('各干扰弹的单独有效遮蔽时间对比');
198. grid on;
199. xlim([0.5, 3.5]);
200. xticks(1:3);
201. xticklabels({'干扰弹 1', '干扰弹 2', '干扰弹 3'});
202.
203.
204. % 三维轨迹可视化
205. figure('Position', [100, 100, 1200, 900], 'Color', 'white');
206. set(gcf, 'DefaultAxesFontSize', 12, 'DefaultTextFontSize', 12);
207.
208. % 配色方案
209. trajectory_colors = {[0, 0.45, 0.74], [0.85, 0.33, 0.1]};
210. element_colors = {[0.93, 0.69, 0.13], [0.49, 0.18, 0.56], [0.47, 0.67, 0.19]};
211.
212. % 绘制导弹轨迹
213. h_missile = plot3(missile_path(:, 1), missile_path(:, 2), missile_path(:, 3), ...
214.     'Color', trajectory_colors{1}, 'LineWidth', 2.5, ...
215.     'Marker', 'o', 'MarkerSize', 5, 'MarkerIndices', 1:50:length(missile_path), ...
216.     'MarkerFaceColor', trajectory_colors{1});
217. hold on;
218.
219. % 绘制无人机轨迹
220. h_drone = plot3(drone_flight_path(:, 1), drone_flight_path(:, 2), drone_flight_path(:, 3), ...
221.     'Color', trajectory_colors{2}, 'LineWidth', 2, ...
222.     'Marker', '^', 'MarkerSize', 5, 'MarkerIndices', 1:30:length(drone_flight_path), ...

```

```

223.     'MarkerFaceColor', trajectory_colors{2});
224.
225. % 绘制圆柱假目标
226. [cyl_X, cyl_Y, cyl_Z] = cylinder(target_cylinder_radius, 50);
227. cyl_Z = cyl_Z * target_cylinder_height;
228. cyl_X = cyl_X + fake_target_base(1);
229. cyl_Y = cyl_Y + fake_target_base(2);
230. cyl_Z = cyl_Z + fake_target_base(3);
231. h_fake_target = surf(cyl_X, cyl_Y, cyl_Z, 'FaceColor', element_colors{1}, ...
232.     'FaceAlpha', 0.3, 'EdgeColor', 'none', ...
233.     'FaceLighting', 'gouraud', 'SpecularStrength', 0.5);
234.
235. % 绘制真目标
236. h_real_target = plot3(true_target_pos(1), true_target_pos(2), true_target_pos(3)
    ), 'ko', ...
237.     'MarkerSize', 12, 'MarkerFaceColor', element_colors{2}, ...
238.     'MarkerEdgeColor', 'k', 'LineWidth', 1.5);
239.
240. % 绘制投放点和起爆点
241. release_coords = [release_position_1; release_position_2; release_position_3];
242. explosion_coords = [explosion_position_1; explosion_position_2; explosion_posit
    ion_3];
243.
244. h_release = plot3(release_coords(:,1), release_coords(:,2), release_coords(:,3)
    , ...
245.     'o', 'Color', element_colors{3}, 'MarkerSize', 10, ...
246.     'MarkerFaceColor', element_colors{3}, 'LineWidth', 1.5);
247.
248. h_explosion = plot3(explosion_coords(:,1), explosion_coords(:,2), explosion_coo
    rds(:,3), ...
249.     'p', 'Color', element_colors{3}, 'MarkerSize', 12, ...
250.     'MarkerFaceColor', element_colors{3}, 'LineWidth', 1.5);
251.
252. % 绘制烟幕云团
253. h_clouds = gobjects(0);
254. if ~isempty(shield_time_intervals)
255.     mid_time_idx = find(time_sequence >= (shield_time_intervals(1,1) + shield_t
    ime_intervals(1,2))/2, 1);
256.     if ~isempty(mid_time_idx)
257.         for cloud_idx = 1:3
258.             if cloud_existence_all(mid_time_idx, cloud_idx) == 1
259.                 start_coord_idx = (cloud_idx-1)*3 + 1;
260.                 cloud_center_pos = cloud_centers_all(mid_time_idx, start_coord
    idx:start_coord_idx+2);

```

```

261.
262.         [sphere_x, sphere_y, sphere_z] = ellipsoid(0, 0, 0, smoke_spher
           e_radius, smoke_sphere_radius*1.2, smoke_sphere_radius*0.8, 30);
263.         sphere_x = sphere_x + cloud_center_pos(1);
264.         sphere_y = sphere_y + cloud_center_pos(2);
265.         sphere_z = sphere_z + cloud_center_pos(3);
266.         h_cloud = surf(sphere_x, sphere_y, sphere_z, 'FaceAlpha', 0.25,
           ...
267.                     'EdgeColor', 'none', 'FaceColor', element_colors{3}, ...
268.                     'FaceLighting', 'gouraud');
269.         h_clouds(end+1) = h_cloud;
270.     end
271. end
272.
273. % 绘制视线连接
274. missile_mid_pos = missile_path(mid_time_idx, :);
275. for point_idx = 1:size(critical_points, 1)
276.     if intersection_results(mid_time_idx, point_idx)
277.         h_line = plot3([missile_mid_pos(1), critical_points(point_idx,
           1)], ...
278.                        [missile_mid_pos(2), critical_points(point_idx, 2)], ...
279.                        [missile_mid_pos(3), critical_points(point_idx, 3)], ...
280.                        'g-', 'LineWidth', 1.2);
281.     else
282.         h_line = plot3([missile_mid_pos(1), critical_points(point_idx,
           1)], ...
283.                        [missile_mid_pos(2), critical_points(point_idx, 2)], ...
284.                        [missile_mid_pos(3), critical_points(point_idx, 3)], ...
285.                        'r--', 'LineWidth', 1.2);
286.     end
287. end
288. end
289. end
290.
291. % 图形属性设置
292. grid on; box on;
293. xlabel('\bfX (m)', 'FontSize', 14);
294. ylabel('\bfY (m)', 'FontSize', 14);
295. zlabel('\bfZ (m)', 'FontSize', 14);
296. title('\bf 三无人机协同投放策略三维轨迹可视化', 'FontSize', 16);
297.
298. % 图例配置
299. legend_components = {
300.     h_missile, '导弹轨迹'

```

```

301.     h_drone, '无人机轨迹'
302.     h_fake_target, '假目标'
303.     h_real_target, '真目标'
304.     h_release, '投放点'
305.     h_explosion, '起爆点'
306. };
307.
308. % 添加烟幕云团图例
309. if ~isempty(h_clouds) && isvalid(h_clouds(1))
310.     legend_components(end+1, :) = {h_clouds(1), '烟幕云团'};
311. end
312.
313. % 添加视线图例
314. if exist('h_line', 'var') && isvalid(h_line)
315.     legend_components(end+1, :) = {h_line, '视线'};
316. end
317.
318. % 生成图例
319. legend_handles_list = legend_components(:, 1);
320. legend_labels_list = legend_components(:, 2);
321.
322. h_legend = legend([legend_handles_list{:}], legend_labels_list, ...
323.     'Location', 'northeast', 'FontSize', 10);
324. set(h_legend, 'Box', 'off', 'Color', [0.95, 0.95, 0.95]);
325.
326. axis equal tight;
327. view(45, 30);
328. light('Position', [1, 0, 0.5], 'Style', 'infinite');
329. lighting gouraud;
330. material dull;
331. set(gca, 'Color', [0.95, 0.95, 0.98], 'GridColor', [0.5, 0.5, 0.5], ...
332.     'GridAlpha', 0.3, 'LineWidth', 1.2);
333.
334. % 保存图像
335. exportgraphics(gcf, '优化_三无人机三维轨迹.png', 'Resolution', 400);
336.
337. % 遮蔽状态时间变化图
338. figure('Position', [100, 100, 1200, 600], 'Color', 'white');
339. set(gcf, 'DefaultAxesFontSize', 12, 'DefaultTextFontSize', 12);
340.
341. % 绘制遮蔽状态阶梯图
342. h_stairs = stairs(time_sequence, shield_effectiveness, 'Color', [0, 0.4, 0], 'L
    ineWidth', 3);
343. hold on;

```

```

344.
345. % 时间区间可视化配色
346. cloud_colors = {[0, 0.4, 0], [0.6, 0, 0.4], [0, 0.4, 0.6]};
347. h_clouds = gobjects(1, 3);
348.
349. explosion_times = [best_explosion_time_1, best_explosion_time_2, best_explosion
    _time_3];
350. for cloud_idx = 1:3
351.     cloud_start_time = explosion_times(cloud_idx);
352.     cloud_end_time = cloud_start_time + smoke_active_time;
353.     active_times = time_sequence(time_sequence >= cloud_start_time & time_seque
        nce <= cloud_end_time);
354.
355.     if ~isempty(active_times)
356.         h_clouds(cloud_idx) = area([active_times(1), active_times(end)], [1, 1]
            , ...
357.             'BaseValue', 0, 'FaceColor', cloud_colors{cloud_idx}, ...
358.             'FaceAlpha', 0.2, 'EdgeColor', 'none');
359.     end
360. end
361.
362. % 标记有效遮蔽区间
363. h_effective = gobjects(0);
364. if ~isempty(shield_time_intervals)
365.     for interval_idx = 1:size(shield_time_intervals, 1)
366.         interval_duration = shield_time_intervals(interval_idx,2) - shield_time
            _intervals(interval_idx,1);
367.
368.         h_eff = rectangle('Position', [shield_time_intervals(interval_idx,1), 0
            .8, interval_duration, 0.15], ...
369.             'FaceColor', cloud_colors{interval_idx}, 'EdgeColor', 'none');
370.         h_effective(end+1) = h_eff;
371.     end
372. end
373.
374. % 标记关键时间点
375. h_release = gobjects(1, 3);
376. h_explosion = gobjects(1, 3);
377.
378. release_times = [best_release_time_1, best_release_time_2, best_release_time_3]
    ;
379. for time_idx = 1:3
380.     % 投放时间标记

```

```

381.     h_release(time_idx) = line([release_times(time_idx), release_times(time_idx
    )], [0, 1], 'Color', cloud_colors{time_idx}, ...
382.         'LineStyle', '--', 'LineWidth', 1.5);
383.
384.     % 起爆时间标记
385.     h_explosion(time_idx) = line([explosion_times(time_idx), explosion_times(ti
    me_idx)], [0, 1], 'Color', cloud_colors{time_idx}, ...
386.         'LineStyle', '-.', 'LineWidth', 1.5);
387. end
388.
389. % 图形属性设置
390. xlabel('\bf 时间 (s)', 'FontSize', 14);
391. ylabel('\bf 遮蔽状态', 'FontSize', 14);
392. title('\bf 三无人机协同策略下的有效遮蔽状态随时间变化', 'FontSize', 16);
393. ylim([0, 1.05]);
394. grid on;
395. set(gca, 'YTick', [0, 1], 'YTickLabel', {'无效', '有效'}, ...
396.     'Color', [0.98, 0.98, 1], 'GridColor', [0.85, 0.85, 0.85]);
397.
398. % 图例配置
399. legend_handles = [];
400. legend_labels = {};
401.
402. % 添加遮蔽状态
403. legend_handles(end+1) = h_stairs;
404. legend_labels{end+1} = '有效遮蔽状态';
405.
406. % 添加云团有效期
407. for cloud_idx = 1:3
408.     if isempty(h_clouds(cloud_idx)) && ~isempty(h_clouds(cloud_idx))
409.         legend_handles(end+1) = h_clouds(cloud_idx);
410.         legend_labels{end+1} = '烟幕云团有效期';
411.         break;
412.     end
413. end
414.
415. % 添加遮蔽区间
416. if ~isempty(h_effective)
417.     legend_handles(end+1) = h_effective(1);
418.     legend_labels{end+1} = '有效遮蔽区间';
419. end
420.
421. % 添加投放时间
422. if ~isempty(h_release)

```



```

423.     legend_handles(end+1) = h_release(1);
424.     legend_labels{end+1} = '投放时间';
425. end
426.
427. % 添加起爆时间
428. if ~isempty(h_explosion)
429.     legend_handles(end+1) = h_explosion(1);
430.     legend_labels{end+1} = '起爆时间';
431. end
432.
433. % 生成图例
434. h_legend_final = legend(legend_handles, legend_labels, ...
435.     'Location', 'northeast', 'FontSize', 10);
436. set(h_legend_final, 'Box', 'off', 'Color', [0.95, 0.95, 0.95]);
437.
438. uistack(h_legend_final, 'top');
439.
440. % 保存图像
441. exportgraphics(gcf, '优化_三无人机遮蔽状态.png', 'Resolution', 400);
442.
443. function shield_time = compute_triple_bomb_shield_time(params, ...
444.     missile_start, missile_dir, missile_vel, drone_start, gravity, cloud_radius
445.     , sink_velocity, cloud_duration, ~)
446.
447.     flight_angle = params(1);
448.     flight_speed = params(2);
449.     release_time_1 = params(3);
450.     delay_time_1 = params(4);
451.     time_gap_21 = params(5);
452.     release_time_2 = release_time_1 + time_gap_21;
453.     delay_time_2 = params(6);
454.     time_gap_32 = params(7);
455.     release_time_3 = release_time_2 + time_gap_32;
456.     delay_time_3 = params(8);
457.
458.     explosion_time_1 = release_time_1 + delay_time_1;
459.     explosion_time_2 = release_time_2 + delay_time_2;
460.     explosion_time_3 = release_time_3 + delay_time_3;
461.
462.     % 飞行方向向量
463.     drone_direction = [cos(flight_angle), sin(flight_angle), 0];
464.
465.     % 计算各干扰弹位置

```

```

466.     release_pos_1 = drone_start + flight_speed * release_time_1 * drone_directi
        on;
467.     explosion_pos_1 = release_pos_1 + delay_time_1 * flight_speed * drone_direc
        tion - 0.5 * gravity * delay_time_1^2 * [0, 0, 1];
468.
469.     release_pos_2 = drone_start + flight_speed * release_time_2 * drone_directi
        on;
470.     explosion_pos_2 = release_pos_2 + delay_time_2 * flight_speed * drone_direc
        tion - 0.5 * gravity * delay_time_2^2 * [0, 0, 1];
471.
472.     release_pos_3 = drone_start + flight_speed * release_time_3 * drone_directi
        on;
473.     explosion_pos_3 = release_pos_3 + delay_time_3 * flight_speed * drone_direc
        tion - 0.5 * gravity * delay_time_3^2 * [0, 0, 1];
474.
475.     time_steps = length(time_array);
476.
477.     missile_positions = zeros(time_steps, 3);
478.     shield_active = zeros(time_steps, 1);
479.     cloud_status = zeros(time_steps, 3);
480.     cloud_centers = zeros(time_steps, 9);
481.
482.     for step_idx = 1:time_steps
483.         current_time = time_array(step_idx);
484.
485.         % 导弹位置计算
486.         missile_positions(step_idx, :) = missile_start + missile_vel * current_
            time * missile_dir;
487.
488.         % 检查各云团状态
489.         clouds_active = false(1, 3);
490.         current_cloud_centers = zeros(1, 9);
491.
492.         % 第一枚干扰弹云团
493.         if current_time >= explosion_time_1 && current_time <= explosion_time_1
            + cloud_duration
494.             clouds_active(1) = true;
495.             current_cloud_centers(1:3) = explosion_pos_1 - sink_velocity * (cur
                rent_time - explosion_time_1) * [0, 0, 1];
496.         end
497.
498.         % 第二枚干扰弹云团
499.         if current_time >= explosion_time_2 && current_time <= explosion_time_2
            + cloud_duration

```

```

500.         clouds_active(2) = true;
501.         current_cloud_centers(4:6) = explosion_pos_2 - sink_velocity * (current_time - explosion_time_2) * [0, 0, 1];
502.     end
503.
504.     % 第三枚干扰弹云团
505.     if current_time >= explosion_time_3 && current_time <= explosion_time_3 + cloud_duration
506.         clouds_active(3) = true;
507.         current_cloud_centers(7:9) = explosion_pos_3 - sink_velocity * (current_time - explosion_time_3) * [0, 0, 1];
508.     end
509.
510.     cloud_status(step_idx, :) = clouds_active;
511.     cloud_centers(step_idx, :) = current_cloud_centers;
512.
513.     % 检查遮蔽效果
514.     point_intersections = zeros(1, size(detection_points, 1));
515.
516.     for bomb_idx = 1:3
517.         if clouds_active(bomb_idx)
518.             center_start_idx = (bomb_idx-1)*3 + 1;
519.             current_cloud_center = cloud_centers(step_idx, center_start_idx:center_start_idx+2);
520.
521.             for point_idx = 1:size(detection_points, 1)
522.                 has_intersection = check_line_sphere_intersection(missile_positions(step_idx,:), detection_points(point_idx,:), current_cloud_center, cloud_radius);
523.                 point_intersections(point_idx) = point_intersections(point_idx) || has_intersection;
524.             end
525.         end
526.     end
527.
528.     shield_active(step_idx) = all(point_intersections == true);
529. end
530. shield_time = sum(shield_active) * time_array(2);
531. end
532.
533. % 三弹遮蔽详情分析函数
534. function [shield_duration, shield_intervals, missile_trajectory, cloud_centers_all, shield_effectiveness, cloud_existence_all, intersection_results] = ...

```

```

535.     analyze_triple_bomb_shielding(params, missile_start, missile_dir, missile_v
      el, drone_start, gravity, cloud_radius, sink_velocity, ...
536.     cloud_duration, detection_points, time_array, ~)
537.
538.     flight_angle = params(1);
539.     flight_speed = params(2);
540.     release_time_1 = params(3);
541.     delay_time_1 = params(4);
542.     release_time_2 = release_time_1 + params(5);
543.     delay_time_2 = params(6);
544.     release_time_3 = release_time_2 + params(7);
545.     delay_time_3 = params(8);
546.
547.     explosion_time_1 = release_time_1 + delay_time_1;
548.     explosion_time_2 = release_time_2 + delay_time_2;
549.     explosion_time_3 = release_time_3 + delay_time_3;
550.
551.     drone_direction = [cos(flight_angle), sin(flight_angle), 0];
552.
553.     % 计算各干扰弹关键位置
554.     release_pos_1 = drone_start + flight_speed * release_time_1 * drone_directi
      on;
555.     explosion_pos_1 = release_pos_1 + delay_time_1 * flight_speed * drone_direc
      tion - 0.5 * gravity * delay_time_1^2 * [0, 0, 1];
556.
557.     release_pos_2 = drone_start + flight_speed * release_time_2 * drone_directi
      on;
558.     explosion_pos_2 = release_pos_2 + delay_time_2 * flight_speed * drone_direc
      tion - 0.5 * gravity * delay_time_2^2 * [0, 0, 1];
559.
560.     release_pos_3 = drone_start + flight_speed * release_time_3 * drone_directi
      on;
561.     explosion_pos_3 = release_pos_3 + delay_time_3 * flight_speed * drone_direc
      tion - 0.5 * gravity * delay_time_3^2 * [0, 0, 1];
562.
563.     time_steps = length(time_array);
564.
565.     missile_trajectory = zeros(time_steps, 3);
566.     shield_effectiveness = zeros(time_steps, 1);
567.     cloud_existence_all = zeros(time_steps, 3);
568.     cloud_centers_all = zeros(time_steps, 9);
569.     intersection_results = zeros(time_steps, size(detection_points, 1));
570.
571.     for step_idx = 1:time_steps

```

```

572.         current_time = time_array(step_idx);
573.
574.         % 导弹轨迹计算
575.         missile_trajectory(step_idx, :) = missile_start + missile_vel * current
            _time * missile_dir;
576.
577.         % 检查各云团状态
578.         clouds_active = false(1, 3);
579.         current_cloud_centers = zeros(1, 9);
580.
581.         % 第一枚干扰弹云团
582.         if current_time >= explosion_time_1 && current_time <= explosion_time_1
            + cloud_duration
583.             clouds_active(1) = true;
584.             current_cloud_centers(1:3) = explosion_pos_1 - sink_velocity * (cur
                rent_time - explosion_time_1) * [0, 0, 1];
585.         end
586.
587.         % 第二枚干扰弹云团
588.         if current_time >= explosion_time_2 && current_time <= explosion_time_2
            + cloud_duration
589.             clouds_active(2) = true;
590.             current_cloud_centers(4:6) = explosion_pos_2 - sink_velocity * (cur
                rent_time - explosion_time_2) * [0, 0, 1];
591.         end
592.
593.         % 第三枚干扰弹云团
594.         if current_time >= explosion_time_3 && current_time <= explosion_time_3
            + cloud_duration
595.             clouds_active(3) = true;
596.             current_cloud_centers(7:9) = explosion_pos_3 - sink_velocity * (cur
                rent_time - explosion_time_3) * [0, 0, 1];
597.         end
598.
599.         cloud_existence_all(step_idx, :) = clouds_active;
600.         cloud_centers_all(step_idx, :) = current_cloud_centers;
601.
602.         % 遮蔽效果判断
603.         point_intersections = zeros(1, size(detection_points, 1));
604.
605.         for bomb_idx = 1:3
606.             if clouds_active(bomb_idx)
607.                 center_start_idx = (bomb_idx-1)*3 + 1;

```

```

608.         current_cloud_center = cloud_centers_all(step_idx, center_start
_idx:center_start_idx+2);
609.
610.         for point_idx = 1:size(detection_points, 1)
611.             has_intersection = check_line_sphere_intersection(missile_t
rajectory(step_idx,:), detection_points(point_idx,:), current_cloud_center, cloud_
radius);
612.             point_intersections(point_idx) = point_intersections(point_
idx) || has_intersection;
613.         end
614.     end
615. end
616.
617.     shield_effectiveness(step_idx) = all(point_intersections == true);
618.     intersection_results(step_idx, :) = point_intersections;
619. end
620.
621. % 计算连续遮蔽时间段
622.     shield_indices = find(shield_effectiveness == 1);
623.     if isempty(shield_indices)
624.         shield_duration = 0;
625.         shield_intervals = [];
626.     else
627.         shield_intervals = [];
628.         start_time = time_array(shield_indices(1));
629.         prev_index = shield_indices(1);
630.
631.         for idx = 2:length(shield_indices)
632.             if shield_indices(idx) - prev_index > 1
633.                 end_time = time_array(prev_index);
634.                 shield_intervals = [shield_intervals; start_time, end_time];
635.                 start_time = time_array(shield_indices(idx));
636.             end
637.             prev_index = shield_indices(idx);
638.         end
639.
640.         end_time = time_array(shield_indices(end));
641.         shield_intervals = [shield_intervals; start_time, end_time];
642.
643.         shield_duration = sum(shield_intervals(:, 2) - shield_intervals(:, 1));
644.
645.         disp('有效遮蔽时间区间: ');
646.         for segment = 1:size(shield_intervals, 1)

```

```

647.         fprintf('区间 %d: %.2f 秒 - %.2f 秒, 持续%.2f 秒\n', segment, shield_intervals(segment, 1), shield_intervals(segment, 2), shield_intervals(segment, 2) - shield_intervals(segment, 1));
648.     end
649.     fprintf('总有效遮蔽时长: %.2f 秒\n', shield_duration);
650. end
651. end
652.
653. % 线段与球体相交检测函数
654. function is_intersected = check_line_sphere_intersection(point1, point2, sphere_center, sphere_radius)
655.     distance1 = norm(point1 - sphere_center);
656.     distance2 = norm(point2 - sphere_center);
657.
658.     % 检查端点是否在球内
659.     if distance1 <= sphere_radius || distance2 <= sphere_radius
660.         is_intersected = true;
661.         return;
662.     end
663.
664.     line_vector = point2 - point1;
665.
666.     center_to_p1 = point1 - sphere_center;
667.     unit_direction = line_vector / norm(line_vector);
668.     perpendicular_distance = sqrt(dot(center_to_p1, center_to_p1) - dot(center_to_p1, unit_direction)^2);
669.
670.     if perpendicular_distance < sphere_radius && dot(center_to_p1, line_vector) < 0
671.         is_intersected = true;
672.     else
673.         is_intersected = false;
674.     End
675. end

```

Q4.mlx

该代码是 MATLAB 语言编写的，用于求解第 4 问及相关绘图。

```

1. missile_pos(t_idx, :) = missile_init_pos + missile_speed * current_time * missile_direction;
2.
3. cloud_exists = false(1, 3);
4. current_cloud_centers = zeros(1, 9);
5.

```

```

6. if current_time >= explosion_time1 && current_time <= explosion_time1
    + cloud_duration
7.     cloud_exists(1) = true;
8.     current_cloud_centers(1:3) = explosion_point1 - cloud_sink_speed
    * (current_time - explosion_time1) * [0, 0, 1];
9. end
10.
11. if current_time >= explosion_time2 && current_time <= explosion_time2
    + cloud_duration
12.     cloud_exists(2) = true;
13.     current_cloud_centers(4:6) = explosion_point2 - cloud_sink_speed
    * (current_time - explosion_time2) * [0, 0, 1];
14. end
15.
16. if current_time >= explosion_time3 && current_time <= explosion_time3
    + cloud_duration
17.     cloud_exists(3) = true;
18.     current_cloud_centers(7:9) = explosion_point3 - cloud_sink_speed
    * (current_time - explosion_time3) * [0, 0, 1];
19. end
20.
21. cloud_status(t_idx, :) = cloud_exists;
22. cloud_centers(t_idx, :) = current_cloud_centers;
23.
24. is_shielded = false;
25. intersection_flags = zeros(1, size(key_points, 1));
26.
27. for bomb_idx = 1:3
28.     if cloud_exists(bomb_idx)
29.         all_points_covered = true;
30.         center_idx = (bomb_idx-1)*3 + 1;
31.         current_cloud_center = cloud_centers(t_idx, center_idx:center
            _idx+2);
32.
33.         for point_idx = 1:size(key_points, 1)
34.             has_intersection = checkLineSphereIntersection(missile_po
                s(t_idx,:), key_points(point_idx,:), current_cloud_center, cloud_radi
                    us);
35.             intersection_flags(point_idx) = intersection_flags(point_
                idx) || has_intersection;
36.             if ~has_intersection
37.                 all_points_covered = false;
38.                 break;
39.             end

```



```

40.         end
41.
42.         if all_points_covered
43.             is_shielded = true;
44.             break;
45.         end
46.     end
47. End

```

Q5.m

该代码是 MATLAB 语言编写的，用于求解第 5 问及相关绘图。

```

1. %% 增强型可视化分析
2. figure('Position', [200, 200, 900, 550], 'Color', [0.96 0.96 0.96]);
3.
4. % 创建子图布局
5. % subplot(2,1,1); % 主分析图
6.
7. % 水平堆叠条形图
8. hb = barh(missile_times, 0.7,...
9.     'FaceColor', 'flat',...
10.    'EdgeColor', [0.3 0.3 0.3],...
11.    'LineWidth', 1.2);
12. hb.CData = [0.1 0.3 0.5; 0.2 0.4 0.6; 0.3 0.5 0.7];
13.
14. % 数据标注优化
15. for i = 1:3
16.     text(missile_times(i)*0.95, i,...
17.         sprintf('%.2f s', missile_times(i)),...
18.         'HorizontalAlignment', 'right',...
19.         'FontSize', 10,...
20.         'Color', [1 1 1],...
21.         'FontWeight', 'bold');
22. end
23.
24. % 参考线及背景
25. yline(1:3, 'Color', [0.9 0.9 0.9], 'LineWidth', 0.5);
26. xline(avg_value, '--', 'Color', [0.8 0.2 0.2], 'LineWidth', 1.5);
27. text(avg_value*1.02, 3.2,...
28.     sprintf('均值: %.2f s', avg_value),...
29.     'Color', [0.8 0.2 0.2],...
30.     'FontSize', 9);
31.
32. % 坐标轴设置

```

```

33. set(gca, 'FontName', 'Arial',...
34.     'FontSize', 10,...
35.     'YTick', 1:3,...
36.     'YTickLabel', {'M1 (X=20000)', 'M2 (X=19000)', 'M3 (X=18000)'},...
37.     'XGrid', 'on',...
38.     'GridLineStyle', ':',...
39.     'GridAlpha', 0.2);
40. xlabel('有效遮蔽时间 (s)');
41. title('导弹遮蔽效能分析 (水平视图)', 'FontSize', 12);
42.
43.
44. %% 输出设置
45. exportgraphics(gcf, 'enhanced_coverage_analysis.png',...
46.     'Resolution', 600, 'BackgroundColor', 'none');
47.
48. %% 无人机贡献占比饼图
49. figure('Position', [200, 200, 800, 800], 'Color', [0.98 0.98 0.98]);
50.
51. % 配色方案（与条形图保持一致）
52. uav_colors = [
53.     0.2 0.5 0.8; % FY1 - 蓝
54.     0.8 0.4 0.2; % FY2 - 橙
55.     0.3 0.7 0.5; % FY3 - 绿
56.     0.7 0.3 0.6; % FY4 - 紫
57.     0.9 0.6 0.1 % FY5 - 金
58. ];
59.
60. % 计算百分比
61. total = sum(UAV_contributions);
62. percentages = UAV_contributions/total*100;
63.
64. % 绘制 3D 饼图
65. p = pie3(UAV_contributions, ...
66.     {'FY1', 'FY2', 'FY3', 'FY4', 'FY5'});
67. %% 修正后的标签设置方法
68. % 获取所有文本对象
69. textObjs = findobj(p, 'Type', 'text');
70.
71. % 设置文本属性
72. set(textObjs, ...
73.     'FontSize', 11, ...
74.     'FontWeight', 'bold', ...
75.     'FontName', 'Arial');
76.

```

```

77. % 设置颜色
78. for i = 1:num_UAVs
79.     set(p(2*i-1), 'FaceColor', uav_colors(i,:), 'EdgeColor', 'none');
80. end
81.
82. % 添加数值标签
83. textObjs = findobj(p, 'Type', 'text');
84. for i = 1:num_UAVs
85.     textObjs(i).String = sprintf('%s\n%.2fs (%.1f%%)', ...
86.         textObjs(i).String, UAV_contributions(i), percentages(i));
87. end
88.
89. % 标题设置
90. title('各无人机对总遮蔽时间的贡献占比', ...
91.     'FontSize', 14, 'FontWeight', 'bold', 'FontName', 'Microsoft YaHei');
92.
93. % 图例
94. legend({'FY1', 'FY2', 'FY3', ...
95.     'FY4', 'FY5'}, ...
96.     'Location', 'eastoutside', 'FontSize', 10);
97.
98. % 导出设置
99. exportgraphics(gcf, 'uav_contribution_pie.png', ...
100.     'Resolution', 600, 'BackgroundColor', 'none');
101.
102. % 计算每个时间点各导弹的遮蔽状态
103. % [~, ~, ~, is_effective] = evaluate_combined_solution_with_status(UAV_params,
104.     r_missiles_0, e_missiles, v_missile, r_UAVs_0, g, R_cloud, v_sink, cloud_lifetime,
105.     key_points, t_range, dt);
106.
107. %% 导弹遮蔽效果可视化（参考图样式）
108. figure('Position', [100, 100, 1200, 800], 'Color', [1 1 1]);
109.
110. % 配色方案（与参考图保持一致）
111. missile_colors = {
112.     [0.2 0.6 1.0], % M1 - 蓝色
113.     [0.1 0.8 0.4], % M2 - 绿色
114.     [0.9 0.3 0.3] % M3 - 红色
115. };
116.
117. for j = 1:3
118.     hold on;

```

```

119.      % 绘制遮蔽状态曲线（二进制阶梯状）
120.      stairs(t_range, is_effective(:, j), 'Color', missile_colors{j}, 'LineWidth'
    , 2.5);
121.
122.      % 绘制有效遮蔽区间（彩色填充）
123.      if ~isempty(effective_intervals{j})
124.          for k = 1:size(effective_intervals{j}, 1)
125.              interval_start = effective_intervals{j}(k, 1);
126.              interval_end = effective_intervals{j}(k, 2);
127.
128.              % 填充有效区间
129.              fill([interval_start, interval_end, interval_end, interval_start],.
    ..
130.                  [0, 0, 1, 1], missile_colors{j}, 'FaceAlpha', 0.3, 'EdgeColor'
    , 'none');
131.
132.      % 添加持续时间标注
133.      if k~=3
134.          if (interval_end - interval_start) > 2
135.              text((interval_start + interval_end)/2, 0.5,...
136.                  sprintf('%1fs', interval_end - interval_start),...
137.                  'HorizontalAlignment', 'center',...
138.                  'VerticalAlignment', 'middle',...
139.                  'FontWeight', 'bold',...
140.                  'FontSize', 10,...
141.                  'Color', [0.2 0.2 0.2]);
142.          end
143.      else
144.          if (interval_end - interval_start) > 2
145.              text((interval_start + interval_end)/2, 0.5,...
146.                  sprintf('%1fs', interval_end - interval_start),...
147.                  'HorizontalAlignment', 'center',...
148.                  'VerticalAlignment', 'top',...
149.                  'FontWeight', 'bold',...
150.                  'FontSize', 10,...
151.                  'Color', [0.2 0.2 0.2]);
152.          end
153.      end
154.  end
155. end
156.
157. % 添加导弹到达时间线（红色虚线）
158. plot([est_flight_time(j), est_flight_time(j)], [0, 1.2], '--',...
159.      'Color', [0.8 0.2 0.2], 'LineWidth', 2);

```

```

160.     text(est_flight_time(j), 1.15, '导弹到达',...
161.         'HorizontalAlignment', 'center',...
162.         'VerticalAlignment', 'bottom',...
163.         'Color', [0.8 0.2 0.2],...
164.         'FontWeight', 'bold',...
165.         'FontSize', 11);
166.
167.     % 设置子图属性
168.     set(gca, 'FontSize', 11, 'FontName', 'Microsoft YaHei');
169.     % title(sprintf('导弹 M%d 有效遮蔽状态 (总时
    间: %.1fs)', j, missile_times(j)),...
170.     %     'FontSize', 13, 'FontWeight', 'bold');
171.     xlabel('时间 (s)', 'FontSize', 12);
172.     ylabel('遮蔽状态', 'FontSize', 12);
173.     ylim([-0.1, 1.3]);
174.     xlim([0, max(est_flight_time) * 1.1]);
175.     grid on;
176.
177.     % 设置 Y 轴刻度
178.     yticks([0 0.2 0.4 0.6 0.8 1.0]);
179.     yticklabels({'0', '0.2', '0.4', '0.6', '0.8', '1.0'});
180.
181.     % 为每条曲线添加标签
182.     text(max(t_range)*0.95, 0.8, sprintf('导弹 M%d', j),...
183.         'Color', missile_colors{j},...
184.         'FontSize', 12,...
185.         'FontWeight', 'bold',...
186.         'BackgroundColor', [1 1 1 0.7],...
187.         'EdgeColor', missile_colors{j},...
188.         'Margin', 3);
189. end
190.
191. % 添加整体标题
192. sgtitle('有效遮蔽状态随时间变化
    ', 'FontSize', 16, 'FontWeight', 'bold', 'FontName', 'Microsoft YaHei');
193.
194. % 创建统一图例
195. legend_handles = gobjects(3,1);
196. for j = 1:3
197.     legend_handles(j) = plot(NaN, NaN, 'Color', missile_colors{j}, 'LineWidth',
        2.5);
198. end
199.

```

```
200. legend(legend_handles, {'导弹 M1 遮蔽状态', '导弹 M2 遮蔽状态', '导弹 M3 遮蔽状态'  
    '},...  
201.     'Orientation', 'horizontal',...  
202.     'Position', [0.25 0.02 0.5 0.04],...  
203.     'FontSize', 11,...  
204.     'FontName', 'Microsoft YaHei');  
205.  
206. % 调整子图间距  
207. set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 1, 1]);  
208.  
209. % 导出图像  
210. exportgraphics(gcf, 'missile_coverage_analysis.png', 'Resolution', 300);
```